

NETWORK PROTOCOLS — UDP AND MTFTP

30.1 EFI UDP Protocol

This chapter defines the EFI UDP (User Datagram Protocol) Protocol that interfaces over the EFI IP Protocol, and the EFI MTFTP Protocol interface that is built upon the EFI UDP Protocol. Protocols for version 4 and version 6 of UDP and MTFTP are included.

30.1.1 UDP4 Service Binding Protocol

30.1.1.1 EFI_UDP4_SERVICE_BINDING_PROTOCOL

Summary

The EFI UDPv4 Service Binding Protocol is used to locate communication devices that are supported by an EFI UDPv4 Protocol driver and to create and destroy instances of the EFI UDPv4 Protocol child protocol driver that can use the underlying communications device.

GUID

```
#define EFI_UDP4_SERVICE_BINDING_PROTOCOL_GUID \
    {0x83f01464, 0x99bd, 0x45e5, \
     {0xb3, 0x83, 0xaf, 0x63, 0x05, 0xd8, 0xe9, 0xe6}}
```

Description

A network application that requires basic UDPv4 I/O services can use one of the protocol handler services, such as *BS->LocateHandleBuffer()*, to search for devices that publish a EFI UDPv4 Service Binding Protocol GUID. Each device with a published EFI UDPv4 Service Binding Protocol GUID supports the EFI UDPv4 Protocol and may be available for use.

After a successful call to the *EFI_UDP4_SERVICE_BINDING_PROTOCOL.CreateChild()* function, the newly created child EFI UDPv4 Protocol driver is in an unconfigured state; it is not ready to send and receive data packets.

Before a network application terminates execution every successful call to the *EFI_UDP4_SERVICE_BINDING_PROTOCOL.CreateChild()* function must be matched with a call to the *EFI_UDP4_SERVICE_BINDING_PROTOCOL.DestroyChild()* function.

30.1.2 UDP4 Protocol

30.1.2.1 EFI_UDP4_PROTOCOL

Summary

The EFI UDPv4 Protocol provides simple packet-oriented services to transmit and receive UDP packets.

GUID

```
#define EFI_UDP4_PROTOCOL_GUID \
    {0x3ad9df29, 0x4501, 0x478d, \
     {0xb1, 0xf8, 0x7f, 0x7f, 0xe7, 0x0e, 0x50, 0xf3}}
```

Protocol Interface Structure

```
typedef struct _EFI_UDP4_PROTOCOL {
    EFI_UDP4_GET_MODE_DATA      GetModeData;
    EFI_UDP4_CONFIGURE          Configure;
    EFI_UDP4_GROUPS             Groups;
    EFI_UDP4_ROUTES             Routes;
    EFI_UDP4_TRANSMIT           Transmit;
    EFI_UDP4_RECEIVE            Receive;
    EFI_UDP4_CANCEL             Cancel;
    EFI_UDP4_POLL               Poll;
} EFI_UDP4_PROTOCOL;
```

Parameters

GetModeData

Reads the current operational settings. See the *GetModeData()* function description.

Configure

Initializes, changes, or resets operational settings for the EFI UDPv4 Protocol. See the *Configure()* function description.

Groups

Joins and leaves multicast groups. See the *Groups()* function description.

Routes

Add and deletes routing table entries. See the *Routes()* function description.

Transmit

Queues outgoing data packets into the transmit queue. This function is a nonblocked operation. See the *Transmit()* function description.

Receive

Places a receiving request token into the receiving queue. This function is a nonblocked operation. See the *Receive()* function description.

Cancel

Aborts a pending transmit or receive request. See the *Cancel()* function description.

Poll

Polls for incoming data packets and processes outgoing data packets. See the *Poll()* function description.

Description

The *EFI_UDP4_PROTOCOL* defines an EFI UDPv4 Protocol session that can be used by any network drivers, applications, or daemons to transmit or receive UDP packets. This protocol instance can either be bound to a specified port

as a service or connected to some remote peer as an active client. Each instance has its own settings, such as the routing table and group table, which are independent from each other.

NOTE: *In this document, all IPv4 addresses and incoming/outgoing packets are stored in network byte order. All other parameters in the functions and data structures that are defined in this document are stored in host byte order.*

30.1.2.2 EFI_UDP4_PROTOCOL.GetModeData()

Summary

Reads the current operational settings.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_UDP4_GET_MODE_DATA) (
    IN EFI_UDP4_PROTOCOL           *This,
    OUT EFI_UDP4_CONFIG_DATA       *Udp4ConfigData OPTIONAL,
    OUT EFI_IP4_MODE_DATA          *Ip4ModeData OPTIONAL,
    OUT EFI_MANAGED_NETWORK_CONFIG_DATA *MnpConfigData OPTIONAL,
    OUT EFI_SIMPLE_NETWORK_MODE    *SnpModeData OPTIONAL
);
```

Parameters

This

Pointer to the *EFI_UDP4_PROTOCOL* instance.

Udp4ConfigData

Pointer to the buffer to receive the current configuration data. Type *EFI_UDP4_CONFIG_DATA* is defined in “Related Definitions” below.

Ip4ModeData*

Pointer to the EFI IPv4 Protocol mode data structure. Type *EFI_IP4_MODE_DATA* is defined in *EFI_IP4_PROTOCOL.GetModeData()*.

MnpConfigData

Pointer to the managed network configuration data structure. Type *EFI_MANAGED_NETWORK_CONFIG_DATA* is defined in *EFI_MANAGED_NETWORK_PROTOCOL.GetModeData()*.

SnpModeData

Pointer to the simple network mode data structure. Type *EFI_SIMPLE_NETWORK_MODE* is defined in the *EFI_SIMPLE_NETWORK_PROTOCOL*.

Description

The *GetModeData()* function copies the current operational settings of this EFI UDPv4 Protocol instance into user-supplied buffers. This function is used optionally to retrieve the operational mode data of underlying networks or drivers.

Related Definition

```
/**
 *
 */
// EFI_UDP4_CONFIG_DATA
/**
 *
 */
typedef struct {
    //Receiving Filters
```

(continues on next page)

(continued from previous page)

```

BOOLEAN          AcceptBroadcast;
BOOLEAN          AcceptPromiscuous;
BOOLEAN          AcceptAnyPort;
BOOLEAN          AllowDuplicatePort;
// I/O parameters
UINT8            TypeOfService;
UINT8            TimeToLive;
BOOLEAN          DoNotFragment;
UINT32           ReceiveTimeout;
UINT32           TransmitTimeout;
// Access Point
BOOLEAN          UseDefaultAddress;
EFI_IPv4_ADDRESS StationAddress;
EFI_IPv4_ADDRESS SubnetMask;
UINT16           StationPort;
EFI_IPv4_ADDRESS RemoteAddress;
UINT16           RemotePort;
} EFI_UDP4_CONFIG_DATA;

```

AcceptBroadcast

Set to **TRUE** to accept broadcast UDP packets.

AcceptPromiscuous

Set to **TRUE** to accept UDP packets that are sent to any address.

AcceptAnyPort

Set to **TRUE** to accept UDP packets that are sent to any port.

AllowDuplicatePort

Set to **TRUE** to allow this EFI UDPv4 Protocol child instance to open a port number that is already being used by another EFI UDPv4 Protocol child instance.

TypeOfService

TypeOfService field in transmitted IPv4 packets.

TimeToLive

TimeToLive field in transmitted IPv4 packets.

DoNotFragment

Set to **TRUE** to disable IP transmit fragmentation.

ReceiveTimeout

The receive timeout value (number of microseconds) to be associated with each incoming packet. Zero means do not drop incoming packets.

TransmitTimeout

The transmit timeout value (number of microseconds) to be associated with each outgoing packet. Zero means do not drop outgoing packets.

UseDefaultAddress

Set to **TRUE** to use the default IP address and default routing table. If the default IP address is not available yet, then the underlying EFI IPv4 Protocol driver will use *EFI_IP4_CONFIG2_PROTOCOL* to retrieve the IP address and subnet information. Ignored for incoming filtering if *AcceptPromiscuous* is set to **TRUE**.

StationAddress

The station IP address that will be assigned to this EFI UDPv4 Protocol instance. The EFI UDPv4 and EFI IPv4 Protocol drivers will only deliver incoming packets whose destination matches this IP address exactly. Address 0.0.0.0 is also accepted as a special case in which incoming packets destined to any station IP address are always

delivered. Not used when *UseDefaultAddress* is **TRUE**. Ignored for incoming filtering if *AcceptPromiscuous* is **TRUE**.

SubnetMask

The subnet address mask that is associated with the station address. Not used when *UseDefaultAddress* is **TRUE**.

StationPort

The port number to which this EFI UDPv4 Protocol instance is bound. If a client of the EFI UDPv4 Protocol does not care about the port number, set *StationPort* to zero. The EFI UDPv4 Protocol driver will assign a random port number to transmitted UDP packets. Ignored if *AcceptAnyPort* is set to **TRUE**.

RemoteAddress

The IP address of remote host to which this EFI UDPv4 Protocol instance is connecting. If *RemoteAddress* is not 0.0.0.0, this EFI UDPv4 Protocol instance will be connected to *RemoteAddress*; i.e., outgoing packets of this EFI UDPv4 Protocol instance will be sent to this address by default and only incoming packets from this address will be delivered to client. Ignored for incoming filtering if *AcceptPromiscuous* is **TRUE**.

RemotePort

The port number of the remote host to which this EFI UDPv4 Protocol instance is connecting. If it is not zero, outgoing packets of this EFI UDPv4 Protocol instance will be sent to this port number by default and only incoming packets from this port will be delivered to client. Ignored if *RemoteAddress* is 0.0.0.0 and ignored for incoming filtering if *AcceptPromiscuous* is **TRUE**.

Status Codes Returned

EFI_SUCCESS	The mode data was read.
EFI_NOT_STARTED	When <i>Udp4ConfigData</i> is queried, no configuration data is available because this instance has not been started.
EFI_INVALID_PARAMETER	<i>This</i> is NULL .

30.1.2.3 EFI_UDP4_PROTOCOL.Configure()

Summary

- Initializes, changes, or resets the operational parameters for this instance of the EFI UDPv4 Protocol.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_UDP4_CONFIGURE) (
    IN EFI_UDP4_PROTOCOL          *This,
    IN EFI_UDP4_CONFIG_DATA      *UdpConfigData OPTIONAL
);
```

Parameters

This

Pointer to the *EFI_UDP4_PROTOCOL* instance.

UdpConfigData

Pointer to the buffer to receive the current mode data.

Description

The *Configure()* function is used to do the following:

- Initialize and start this instance of the EFI UDPv4 Protocol.

- Change the filtering rules and operational parameters.
- Reset this instance of the EFI UDPv4 Protocol.

Until these parameters are initialized, no network traffic can be sent or received by this instance. This instance can be also reset by calling *Configure()* with *UdpConfigData* set to *NULL*. Once reset, the receiving queue and transmitting queue are flushed and no traffic is allowed through this instance.

With different parameters in *UdpConfigData*, *Configure()* can be used to bind this instance to specified port.

Status Codes Returned

EFI_SUCCESS	The configuration settings were set, changed, or reset successfully.
EFI_NO_MAPPING	When using a default address, configuration (DHCP, BOOTP, RARP, etc.) is not finished yet.
EFI_INVALID_PARAMETER	One or more following conditions are TRUE : • This is NULL. • <i>UdpConfigData.StationAddress</i> is not a valid unicast IPv4 address. • <i>UdpConfigData.SubnetMask</i> is not a valid IPv4 address mask. The subnet mask must be contiguous. • <i>UdpConfigData.RemoteAddress</i> is not a valid unicast IPv4 address if it is not zero.
EFI_ALREADY_STARTED	The EFI UDPv4 Protocol instance is already started/configured and must be stopped/reset before it can be reconfigured. Only <i>TypeOfService</i> , <i>TimeToLive</i> , <i>DoNotFragment</i> , <i>ReceiveTimeout</i> , and <i>TransmitTimeout</i> can be reconfigured without stopping the current instance of the EFI UDPv4 Protocol.
EFI_ACCESS_DENIED	<i>UdpConfigData.AllowDuplicatePort</i> is FALSE and <i>UdpConfigData.StationPort</i> is already used by other instance.
EFI_OUT_OF_RESOURCES	The EFI UDPv4 Protocol driver cannot allocate memory for this EFI UDPv4 Protocol instance.
EFI_DEVICE_ERROR	An unexpected network or system error occurred and this instance was not opened.

30.1.2.4 EFI_UDP4_PROTOCOL.Groups()

Summary

Joins and leaves multicast groups.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_UDP4_GROUPS) (
    IN EFI_UDP4_PROTOCOL    *This,
    IN BOOLEAN              JoinFlag,
    IN EFI_IPv4_ADDRESS      *MulticastAddress OPTIONAL
);
```

Parameters

This

Pointer to the *EFI_UDP4_PROTOCOL* instance.

JoinFlag

Set to **TRUE** to join a multicast group. Set to **FALSE** to leave one or all multicast groups.

MulticastAddress

Pointer to multicast group address to join or leave.

Description

The *Groups()* function is used to enable and disable the multicast group filtering.

If the *JoinFlag* is **FALSE** and the *MulticastAddress* is **NULL**, then all currently joined groups are left.

Status Codes Returned

EFI_SUCCESS	The operation completed successfully.
EFI_NOT_STARTED	The EFI UDPv4 Protocol instance has not been started.
EFI_NO_MAPPING	When using a default address, configuration (DHCP, BOOTP, RARP, etc.) is not finished yet.
EFI_OUT_OF_RESOURCES	Could not allocate resources to join the group.
EFI_INVALID_PARAMETER	One or more of the following conditions is TRUE: • This is NULL. • JoinFlag is TRUE and MulticastAddress is NULL. • JoinFlag is TRUE and *MulticastAddress is not a valid multicast address.
EFI_ALREADY_STARTED	The group address is already in the group table (when JoinFlag is TRUE).
EFI_NOT_FOUND	The group address is not in the group table (when JoinFlag is FALSE).
EFI_DEVICE_ERROR	An unexpected system or network error occurred.

30.1.2.5 EFI_UDP4_PROTOCOL.Routes()**Summary**

Adds and deletes routing table entries.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_UDP4_ROUTES) (
    IN EFI_UDP4_PROTOCOL      *This,
    IN BOOLEAN                DeleteRoute,
    IN EFI_IPv4_ADDRESS        *SubnetAddress,
    IN EFI_IPv4_ADDRESS        *SubnetMask,
    IN EFI_IPv4_ADDRESS        *GatewayAddress
);
```

Parameters**This**

Pointer to the *EFI_UDP4_PROTOCOL* instance.

DeleteRoute

Set to **TRUE** to delete this route from the routing table. Set to **FALSE** to add this route to the routing table. *DestinationAddress* and *SubnetMask* are used as the key to each route entry.

SubnetAddress

The destination network address that needs to be routed.

SubnetMask

The subnet mask of *SubnetAddress*.

GatewayAddress

The gateway IP address for this route.

Description

The *Routes()* function adds a route to or deletes a route from the routing table.

Routes are determined by comparing the *SubnetAddress* with the destination IP address and arithmetically *AND* -ing it with the *SubnetMask*. The gateway address must be on the same subnet as the configured station address.

The default route is added with *SubnetAddress* and *SubnetMask* both set to 0.0.0.0. The default route matches all destination IP addresses that do not match any other routes.

A zero *GatewayAddress* is a nonroute. Packets are sent to the destination IP address if it can be found in the Address Resolution Protocol (ARP) cache or on the local subnet. One automatic nonroute entry will be inserted into the routing table for outgoing packets that are addressed to a local subnet (gateway address of 0.0.0.0).

Each instance of the EFI UDPv4 Protocol has its own independent routing table. Instances of the EFI UDPv4 Protocol that use the default IP address will also have copies of the routing table provided by the *EFI_IP4_CONFIG2_PROTOCOL*. These copies will be updated automatically whenever the IP driver reconfigures its instances; as a result, the previous modification to these copies will be lost.

NOTE: *There is no way to set up routes to other network interface cards (NICs) because each NIC has its own independent network stack that shares information only through EFI UDP4 Variable.*

Status Codes Returned

EFI_SUCCESS	The operation completed successfully.
EFI_NOT_STARTED	The EFI UDPv4 Protocol instance has not been started.
EFI_NO_MAPPING	When using a default address, configuration (DHCP, BOOTP, RARP, etc.) is not finished yet.
EFI_INVALID_PARAMETER	One or more of the following conditions is TRUE : • <i>This</i> is NULL. • <i>SubnetAddress</i> is NULL. • <i>SubnetMask</i> is NULL. • <i>GatewayAddress</i> is NULL. • <i>SubnetAddress</i> is not a valid subnet address. • <i>SubnetMask</i> is not a valid subnet mask. • <i>GatewayAddress</i> is not a valid unicast IP address.
EFI_OUT_OF_RESOURCES	Could not add the entry to the routing table.
EFI_NOT_FOUND	This route is not in the routing table.
EFI_ACCESS_DENIED	The route is already defined in the routing table.

30.1.2.6 EFI_UDP4_PROTOCOL.Transmit()

Summary

Queues outgoing data packets into the transmit queue.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_UDP4_TRANSMIT) (
    IN EFI_UDP4_PROTOCOL          *This,
    IN EFI_UDP4_COMPLETION_TOKEN *Token
);
```

Parameters

This

Pointer to the *EFI_UDP4_PROTOCOL* instance.

Token

Pointer to the completion token that will be placed into the transmit queue. Type *EFI_UDP4_COMPLETION_TOKEN* is defined in “Related Definitions” below.

Description

The *Transmit()* function places a sending request to this instance of the EFI UDPv4 Protocol, alongside the transmit data that was filled by the user. Whenever the packet in the token is sent out or some errors occur, the *Token.Event* will be signaled and *Token.Status* is updated. Providing a proper notification function and context for the event will enable the user to receive the notification and transmitting status.

Related Definitions

```
/**
 *
 */
// EFI_UDP4_COMPLETION_TOKEN
/**
 *
 */
typedef struct {
    EFI_EVENT          Event;
    EFI_STATUS         Status;
    union {
        EFI_UDP4_RECEIVE_DATA *RxData;
        EFI_UDP4_TRANSMIT_DATA *TxData;
    } Packet;
} EFI_UDP4_COMPLETION_TOKEN;
```

Event

This *Event* will be signaled after the *Status* field is updated by the EFI UDPv4 Protocol driver. The type of *Event* must be *EVT_NOTIFY_SIGNAL*. The Task Priority Level (TPL) of *Event* must be lower than or equal to *TPL_CALLBACK*.

Status

Will be set to one of the following values:

EFI_SUCCESS. The receive or transmit operation completed successfully.

EFI_ABORTED. The receive or transmit was aborted.

EFI_TIMEOUT. The transmit timeout expired.

EFI_NETWORK_UNREACHABLE. The destination network is unreachable. RxData is set to **NULL** in this situation.

EFI_HOST_UNREACHABLE. The destination host is unreachable. RxData is set to **NULL** in this situation.

EFI_PROTOCOL_UNREACHABLE. The UDP protocol is unsupported in the remote system. RxData is set to **NULL** in this situation.

EFI_PORT_UNREACHABLE. No service is listening on the remote port. RxData is set to **NULL** in this situation.

EFI_ICMP_ERROR. Some other Internet Control Message Protocol (ICMP) error report was received. For example, packets are being sent too fast for the destination to receive them and the destination sent an ICMP source quench report. RxData is set to **NULL** in this situation.

EFI_DEVICE_ERROR. An unexpected system or network error occurred.

EFI_NO_MEDIA. There was a media error.

RxData

When this token is used for receiving, *RxData* is a pointer to *EFI_UDP4_RECEIVE_DATA*. Type *EFI_UDP4_RECEIVE_DATA* is defined below.

TxData

When this token is used for transmitting, *TxData* is a pointer to *EFI_UDP4_TRANSMIT_DATA*. Type *EFI_UDP4_TRANSMIT_DATA* is defined below.

The *EFI_UDP4_COMPLETION_TOKEN* structures are used for both transmit and receive operations.

When used for transmitting, the *Event* and *TxData* fields must be filled in by the EFI UDPv4 Protocol client. After the transmit operation completes, the *Status* field is updated by the EFI UDPv4 Protocol and the *Event* is signaled.

- When used for receiving, only the *Event* field must be filled in by the EFI UDPv4 Protocol client. After a packet is received, *RxData* and *Status* are filled in by the EFI UDPv4 Protocol and the *Event* is signaled.
- The ICMP related status codes filled in *Status* are defined as follows:

```

//*****
// UDP4 Token Status definition
//*****
#define EFI_NETWORK_UNREACHABLE    EFIERR(100)
#define EFI_HOST_UNREACHABLE      EFIERR(101)
#define EFI_PROTOCOL_UNREACHABLE  EFIERR(102)
#define EFI_PORT_UNREACHABLE      EFIERR(103)

//*****
// EFI_UDP4_RECEIVE_DATA
//*****
typedef struct {
    EFI_TIME           TimeStamp;
    EFI_EVENT          RecycleSignal;
    EFI_UDP4_SESSION_DATA  UdpSession;
    UINT32             DataLength;
    UINT32             FragmentCount;
    EFI_UDP4_FRAGMENT_DATA  FragmentTable[1];
} EFI_UDP4_RECEIVE_DATA;

```

TimeStamp

Time when the EFI UDPv4 Protocol accepted the packet. *TimeStamp* is zero filled if timestamps are disabled or unsupported

RecycleSignal

Indicates the event to signal when the received data has been processed.

UdpSession

The UDP session data including *SourceAddress*, *SourcePort*, *DestinationAddress*, and *DestinationPort*. Type *EFI_UDP4_SESSION_DATA* is defined below.

DataLength

The sum of the fragment data length.

FragmentCount

Number of fragments. May be zero.

FragmentTable

Array of fragment descriptors. IP and UDP headers are included in these buffers if *ConfigData.RawData* is **TRUE**. Otherwise they are stripped. May be zero. Type *EFI_UDP4_FRAGMENT_DATA* is defined below.

EFI_UDP4_RECEIVE_DATA is filled by the EFI UDPv4 Protocol driver when this EFI UDPv4 Protocol instance receives an incoming packet. If there is a waiting token for incoming packets, the *CompletionToken.Packet.RxData* field is updated to this incoming packet and the *CompletionToken.Event* is signaled. The EFI UDPv4 Protocol client must signal the *RecycleSignal* after processing the packet.

- *FragmentTable* could contain multiple buffers that are not in the continuous memory locations. The EFI UDPv4 Protocol client might need to combine two or more buffers in *FragmentTable* to form their own protocol header.

```
//*****
// EFI_UDP4_SESSION_DATA
//*****
typedef struct {
    EFI_IPv4_ADDRESS    SourceAddress;
    UINT16              SourcePort;
    EFI_IPv4_ADDRESS    DestinationAddress;
    UINT16              DestinationPort;
} EFI_UDP4_SESSION_DATA;
```

SourceAddress

Address from which this packet is sent. If this field is set to zero when sending packets, the address that is assigned in *EFI_UDP4_PROTOCOL.Configure()* is used.

SourcePort

Port from which this packet is sent. It is in host byte order. If this field is set to zero when sending packets, the port that is assigned in *EFI_UDP4_PROTOCOL.Configure()* is used. If this field is set to zero and unbound, a call to *EFI_UDP4_PROTOCOL.Transmit()* will fail.

DestinationAddress

Address to which this packet is sent.

DestinationPort

Port to which this packet is sent. It is in host byte order. If this field is set to zero and unconnected, the call to *EFI_UDP4_PROTOCOL.Transmit()* will fail.

The *EFI_UDP4_SESSION_DATA* is used to retrieve the settings when receiving packets or to override the existing settings of this EFI UDPv4 Protocol instance when sending packets.

```
//*****
// EFI_UDP4_FRAGMENT_DATA
```

(continues on next page)

(continued from previous page)

```

//*****
typedef struct {
    UINT32          FragmentLength;
    VOID            *FragmentBuffer;
}  EFI_UDP4_FRAGMENT_DATA;

```

FragmentLength

Length of the fragment data buffer.

FragmentBuffer

Pointer to the fragment data buffer.

EFI_UDP4_FRAGMENT_DATA allows multiple receive or transmit buffers to be specified. The purpose of this structure is to avoid copying the same packet multiple times.

```

//*****
//  EFI_UDP4_TRANSMIT_DATA
//*****
typedef struct {
    EFI_UDP4_SESSION_DATA    *UdpSessionData;
    EFI_IPv4_ADDRESS          *GatewayAddress;
    UINT32                    DataLength;
    UINT32                    FragmentCount;
    EFI_UDP4_FRAGMENT_DATA    FragmentTable[1];
}  EFI_UDP4_TRANSMIT_DATA;

```

UdpSessionData

If not **NULL**, the data that is used to override the transmitting settings. Type *EFI_UDP4_SESSION_DATA* is defined above.

GatewayAddress

The next-hop address to override the setting from the routing table.

DataLength

Sum of the fragment data length. Must not exceed the maximum UDP packet size.

FragmentCount

Number of fragments.

FragmentTable

Array of fragment descriptors. Type *EFI_UDP4_FRAGMENT_DATA* is defined above.

The EFI UDPv4 Protocol client must fill this data structure before sending a packet. The packet may contain multiple buffers that may be not in a continuous memory location.

Status Codes Returned

EFI_SUCCESS	The data has been queued for transmission.
EFI_NOT_STARTED	This EFI UDPv4 Protocol instance has not been started.
EFI_NO_MAPPING	When using a default address, configuration (DHCP, BOOTP, RARP, etc.) is not finished yet.

continues on next page

Table 30.5 – continued from previous page

EFI_INVALID_PARAMETER	One or more of the following are TRUE: • <i>This</i> is NULL. • <i>Token</i> is NULL. • <i>Token.Event</i> is NULL. • <i>Token.Packet.TxData</i> is NULL. • <i>Token.Packet.TxData.FragmentCount</i> is zero. • <i>Token.Packet.TxData.DataLength</i> is not equal to the sum of fragment lengths. • One or more of the <i>Token.Packet.TxData.FragmentTable[]</i>.<i>FragmentLength</i> fields is zero. • One or more of the <i>Token.Packet.TxData.FragmentTable[]</i>.<i>FragmentBuffer</i> fields is NULL. • <i>Token.Packet.TxData.GatewayAddress</i> is not a unicast IPv4 address if it is not NULL. • <i>Token.Packet.TxData.UdpSessionData.SourceAddress</i> is not a valid unicast IPv4 address or <i>Token.Packet.TxData.UdpSessionData.DestinationAddress</i> is zero if the <i>UdpSessionData</i> is not NULL.
EFI_ACCESS_DENIED	The transmit completion token with the same <i>Token.Event</i> was already in the transmit queue.
EFI_NOT_READY	The completion token could not be queued because the transmit queue is full.
EFI_OUT_OF_RESOURCES	Could not queue the transmit data.
EFI_NOT_FOUND	There is no route to the destination network or address.
EFI_BAD_BUFFER_SIZE	The data length is greater than the maximum UDP packet size. Or the length of the IP header + UDP header + data length is greater than MTU if <i>DoNotFragment</i> is TRUE .
EFI_NO_MEDIA	There was a media error.

30.1.2.7 EFI_UDP4_PROTOCOL.Receive()

Summary

Places an asynchronous receive request into the receiving queue.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_UDP4_RECEIVE) (
    IN EFI_UDP4_PROTOCOL          *This,
    IN EFI_UDP4_COMPLETION_TOKEN *Token
);
```

Parameters

This

Pointer to the *EFI_UDP4_PROTOCOL* instance.

Token

Pointer to a token that is associated with the receive data descriptor. Type *EFI_UDP4_COMPLETION_TOKEN* is defined in *EFI_UDP4_PROTOCOL*.*Transmit()*.

Description

The *Receive()* function places a completion token into the receive packet queue. This function is always asynchronous.

The caller must fill in the *Token.Event* field in the completion token, and this field cannot be **NULL**. When the receive operation completes, the EFI UDPv4 Protocol driver updates the *Token.Status* and *Token.Packet.RxData* fields and the *Token.Event* is signaled. Providing a proper notification function and context for the event will enable the user to receive the notification and receiving status. That notification function is guaranteed to not be re-entered.

Status Codes Returned

EFI_SUCCESS	The receive completion token was cached.
EFI_NOT_STARTED	This EFI UDPv4 Protocol instance has not been started.
EFI_NO_MAPPING	When using a default address, configuration (DHCP, BOOTP, RARP, etc.) is not finished yet.
EFI_INVALID_PARAMETER	One or more of the following conditions is TRUE : • <i>This</i> is NULL. • <i>Token</i> is NULL. • <i>Token.Event</i> is NULL.
EFI_OUT_OF_RESOURCES	The receive completion token could not be queued due to a lack of system resources (usually memory).
EFI_DEVICE_ERROR	An unexpected system or network error occurred. The EFI UDPv4 Protocol instance has been reset to startup defaults.
EFI_ACCESS_DENIED	A receive completion token with the same <i>Token.Event</i> was already in the receive queue.
EFI_NOT_READY	The receive request could not be queued because the receive queue is full.
EFI_NO_MEDIA	There was a media error.

30.1.2.8 EFI_UDP4_PROTOCOL.Cancel()**Summary**

Aborts an asynchronous transmit or receive request.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_UDP4_CANCEL)(
    IN EFI_UDP4_PROTOCOL          *This,
    IN EFI_UDP4_COMPLETION_TOKEN *Token OPTIONAL
);
```

Parameters**This**

Pointer to the *EFI_UDP4_PROTOCOL* instance.

Token

Pointer to a token that has been issued by *EFI_UDP4_PROTOCOL*.*Transmit()* or *EFI_UDP4_PROTOCOL.Receive()*. If **NULL**, all pending tokens are aborted. Type *EFI_UDP4_COMPLETION_TOKEN* is defined in *EFI_UDP4_PROTOCOL.Transmit()*.

Description

The *Cancel()* function is used to abort a pending transmit or receive request. If the token is in the transmit or receive request queues, after calling this function, *Token.Status* will be set to *EFI_ABORTED* and then *Token.Event* will be signaled. If the token is not in one of the queues, which usually means that the asynchronous operation has completed, this function will not signal the token and *EFI_NOT_FOUND* is returned.

Status Codes Returned

<i>EFI_SUCCESS</i>	The asynchronous I/O request was aborted and <i>Token.Event</i> was signaled. When <i>Token</i> is NULL , all pending requests are aborted and their events are signaled.
<i>EFI_INVALID_PARAMETER</i>	<i>This</i> is NULL .
<i>EFI_NOT_STARTED</i>	This instance has not been started.
<i>EFI_NO_MAPPING</i>	When using the default address, configuration (DHCP, BOOTP, RARP, etc.) is not finished yet.
<i>EFI_NOT_FOUND</i>	When <i>Token</i> is not NULL , the asynchronous I/O request was not found in the transmit or receive queue. It has either completed or was not issued by <i>Transmit()</i> and <i>Receive()</i> .

30.1.2.9 EFI_UDP4_PROTOCOL.Poll()**Summary**

Polls for incoming data packets and processes outgoing data packets.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_UDP4_POLL) (
    IN EFI_UDP4_PROTOCOL    *This
);
```

Parameters**This**

Pointer to the *EFI_UDP4_PROTOCOL* instance.

Description

The *Poll()* function can be used by network drivers and applications to increase the rate that data packets are moved between the communications device and the transmit and receive queues.

In some systems, the periodic timer event in the managed network driver may not poll the underlying communications device fast enough to transmit and/or receive all data packets without missing incoming packets or dropping outgoing packets. Drivers and applications that are experiencing packet loss should try calling the *Poll()* function more often.

Status Codes Returned

<i>EFI_SUCCESS</i>	Incoming or outgoing data was processed.
--------------------	--

continues on next page

Table 30.8 – continued from previous page

EFI_INVALID_PARAMETER	This is NULL .
EFI_DEVICE_ERROR	An unexpected system or network error occurred.
EFI_TIMEOUT	Data was dropped out of the transmit and/or receive queue. Consider increasing the polling rate.

30.2 EFI UDPv6 Protocol

This section defines the EFI UDPv6 (User Datagram Protocol version 6) Protocol that interfaces over the EFI IPv6 Protocol.

30.2.1 UDP6 Service Binding Protocol

30.2.1.1 EFI_UDP6_SERVICE_BINDING_PROTOCOL

Summary

The EFI UDPv6 Service Binding Protocol is used to locate communication devices that are supported by an EFI UDPv6 Protocol driver and to create and destroy instances of the EFI UDPv6 Protocol child instance that uses the underlying communications device.

GUID

```
#define EFI_UDP6_SERVICE_BINDING_PROTOCOL_GUID \
    {0x66ed4721, 0x3c98, 0x4d3e, \
     {0x81, 0xe3, 0xd0, 0x3d, 0xd3, 0x9a, 0x72, 0x54}}
```

Descriptopn

A network application that requires basic UDPv6 I/O services can use one of the protocol handler services, such as *BS->LocateHandleBuffer()*, to search for devices that publish a EFI UDPv6 Service Binding Protocol GUID. Each device with a published EFI UDPv6 Service Binding Protocol GUID supports the EFI UDPv6 Protocol and may be available for use.

After a successful call to the *EFI_UDP6_SERVICE_BINDING_PROTOCOL.CreateChild()* function, the newly created child EFI UDPv6 Protocol driver is in an un-configured state; it is not ready to send and receive data packets.

Before a network application terminates execution, every successful call to the *EFI_UDP6_SERVICE_BINDING_PROTOCOL.CreateChild()* function must be matched with a call to the *EFI_UDP6_SERVICE_BINDING_PROTOCOL.DestroyChild()* function.

30.2.2 EFI UDP6 Protocol

30.2.2.1 EFI_UDP6_PROTOCOL

Summary

The EFI UDPv6 Protocol provides simple packet-oriented services to transmit and receive UDP packets.

GUID


```
#define EFI_UDP6_PROTOCOL_GUID \
    {0x4f948815, 0xb4b9, 0x43cb, \
     {0x8a, 0x33, 0x90, 0xe0, 0x60, 0xb3, 0x49, 0x55}}
```

Protocol Interface Structure

```
typedef struct _EFI_UDP6_PROTOCOL {
    EFI_UDP6_GET_MODE_DATA      GetModeData;
    EFI_UDP6_CONFIGURE          Configure;
    EFI_UDP6_GROUPS              Groups;
    EFI_UDP6_TRANSMIT            Transmit;
    EFI_UDP6_RECEIVE             Receive;
    EFI_UDP6_CANCEL              Cancel;
    EFI_UDP6_POLL                Poll;
} EFI_UDP6_PROTOCOL;
```

Parameters

GetModeData

Reads the current operational settings. See the *GetModeData()* function description.

Configure

Initializes, changes, or resets operational settings for the EFI UDPv6 Protocol. See the *Configure()* function description.

Groups

Joins and leaves multicast groups. See the *Groups()* function description.

Transmit

Queues outgoing data packets into the transmit queue. This function is a non-blocked operation. See the *Transmit()* function description.

Receive

Places a receiving request token into the receiving queue. This function is a non-blocked operation. See the *Receive()* function description.

Cancel

Aborts a pending transmit or receive request. See the *Cancel()* function description.

Poll

Polls for incoming data packets and processes outgoing data packets. See the *Poll()* function description.

Description

The *EFI_UDP6_PROTOCOL* defines an EFI UDPv6 Protocol session that can be used by any network drivers, applications, or daemons to transmit or receive UDP packets. This protocol instance can either be bound to a specified port as a service or connected to some remote peer as an active client. Each instance has its own settings, such as group table, that are independent from each other.

Note

Byte Order: In this document, all IPv6 addresses and incoming/outgoing packets are stored in network byte order. All other parameters in the functions and data structures that are defined in this document are stored in host byte order.

30.2.2.2 EFI_UDP6_PROTOCOL.GetModeData()

Summary

Read the current operational settings.

Prototype

```
typedef
EFI_STATUS
(EFI_API *EFI_UDP6_GET_MODE_DATA) (
    IN EFI_UDP6_PROTOCOL          *This,
    OUT EFI_UDP6_CONFIG_DATA      *Udp6ConfigData OPTIONAL,
    OUT EFI_IP6_MODE_DATA         *Ip6ModeData OPTIONAL,
    OUT EFI_MANAGED_NETWORK_CONFIG_DATA *MnpConfigData OPTIONAL,
    OUT EFI_SIMPLE_NETWORK_MODE   *SnpModeData OPTIONAL
);
```

Parameters

This

Pointer to the EFI_UDP6_PROTOCOL instance.

Udp6ConfigData

The buffer in which the current UDP configuration data is returned. Type *EFI_UDP6_CONFIG_DATA* is defined in “Related Definitions” below.

Ip6ModeData

The buffer in which the current EFI IPv6 Protocol mode data is returned. Type *EFI_IP6_MODE_DATA* is defined in *EFI_IP6_PROTOCOL.GetModeData()*.

MnpConfigData

The buffer in which the current managed network configuration data is returned. Type *EFI_MANAGED_NETWORK_CONFIG_DATA* is defined in *EFI_MANAGED_NETWORK_PROTOCOL.GetModeData()*.

SnpModeData

The buffer in which the simple network mode data is returned. Type *EFI_SIMPLE_NETWORK_MODE* is defined in the *EFI_SIMPLE_NETWORK* Protocol.

Description

The *GetModeData()* function copies the current operational settings of this EFI UDPv6 Protocol instance into user-supplied buffers. This function is used optionally to retrieve the operational mode data of underlying networks or drivers.

Related Definition

```
*****
// EFI_UDP6_CONFIG_DATA
//
*****
typedef struct {
    //Receiving Filters

    BOOLEAN    AcceptPromiscuous;
    BOOLEAN    AcceptAnyPort;
    BOOLEAN    AllowDuplicatePort;
```

(continues on next page)

(continued from previous page)

```

//I/O parameters
;

UINT8          TrafficClass;
UINT8          HopLimit;
;
UINT32         ReceiveTimeout;
UINT32         TransmitTimeout;
//Access Point
EFI_IPv6_ADDRESS StationAddress;
UINT16         StationPort;
EFI_IPv6_ADDRESS RemoteAddress;
UINT16         RemotePort;
} EFI_UDP6_CONFIG_DATA;

```

AcceptPromiscuous

Set to **TRUE** to accept UDP packets that are sent to any address.

AcceptAnyPort

Set to **TRUE** to accept UDP packets that are sent to any port.

AllowDuplicatePort

Set to **TRUE** to allow this EFI UDPv6 Protocol child instance to open a port number that is already being used by another EFI UDPv6 Protocol child instance.

TrafficClass

TrafficClass field in transmitted IPv6 packets.

HopLimit

HopLimit field in transmitted IPv6 packets.

ReceiveTimeout

The receive timeout value (number of microseconds) to be associated with each incoming packet. Zero means do not drop incoming packets.

TransmitTimeout

The transmit timeout value (number of microseconds) to be associated with each outgoing packet. Zero means do not drop outgoing packets.

StationAddress

The station IP address that will be assigned to this EFI UDPv6 Protocol instance. The EFI UDPv6 and EFI IPv6 Protocol drivers will only deliver incoming packets whose destination matches this IP address exactly. Address 0:/128 is also accepted as a special case. Under this situation, underlying IPv6 driver is responsible for binding a source address to this EFI IPv6 protocol instance according to source address selection algorithm. Only incoming packet from the selected source address is delivered. This field can be set and changed only when the EFI IPv6 driver is transitioning from the stopped to the started states. If no address is available for selecting, the EFI IPv6 Protocol driver will use *EFI_IP6_CONFIG_PROTOCOL* to retrieve the IPv6 address.

StationPort

The port number to which this EFI UDPv6 Protocol instance is bound. If a client of the EFI UDPv6 Protocol does not care about the port number, set *StationPort* to zero. The EFI UDPv6 Protocol driver will assign a random port number to transmitted UDP packets. Ignored it if *AcceptAnyPort* is **TRUE**.

RemoteAddress

The IP address of remote host to which this EFI UDPv6 Protocol instance is connecting. If *RemoteAddress* is not 0:/128, this EFI UDPv6 Protocol instance will be connected to *RemoteAddress*; i.e., outgoing packets of this

EFI UDPv6 Protocol instance will be sent to this address by default and only incoming packets from this address will be delivered to client. Ignored for incoming filtering if *AcceptPromiscuous* is **TRUE**.

RemotePort

The port number of the remote host to which this EFI UDPv6 Protocol instance is connecting. If it is not zero, outgoing packets of this EFI UDPv6 Protocol instance will be sent to this port number by default and only incoming packets from this port will be delivered to client. Ignored if *RemoteAddress* is 0:/128 and ignored for incoming filtering if *AcceptPromiscuous* is **TRUE**.

Status Codes Returned

EFI_SUCCESS	The mode data was read.
EFI_NOT_STARTED	When <i>Udp6ConfigData</i> is queried, no configuration data is available because this instance has not been started.
EFI_INVALID_PARAMETER	<i>This</i> is NULL .

30.2.2.3 EFI_UDP6_PROTOCOL.Configure()

Summary

Initializes, changes, or resets the operational parameters for this instance of the EFI UDPv6 Protocol.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_UDP6_CONFIGURE) (
    IN EFI_UDP6_PROTOCOL      *This,
    IN EFI_UDP6_CONFIG_DATA   *UdpConfigData OPTIONAL
);
```

Parameters

This

Pointer to the *EFI_UDP6_PROTOCOL* instance.

UdpConfigData

Pointer to the buffer contained the configuration data.

Description

The *Configure()* function is used to do the following:

- Initialize and start this instance of the EFI UDPv6 Protocol.
- Change the filtering rules and operational parameters.
- Reset this instance of the EFI UDPv6 Protocol.

Until these parameters are initialized, no network traffic can be sent or received by this instance. This instance can be also reset by calling *Configure()* with *UdpConfigData* set to **NULL**. Once reset, the receiving queue and transmitting queue are flushed and no traffic is allowed through this instance.

With different parameters in *UdpConfigData*, *Configure()* can be used to bind this instance to specified port.

Status Codes Returned

EFI_SUCCESS	The configuration settings were set, changed, or reset successfully.
-------------	--

continues on next page

Table 30.10 – continued from previous page

EFI_NO_MAPPING	The underlying IPv6 driver was responsible for choosing a source address for this instance, but no source address was available for use.
EFI_INVALID_PARAMETER	One or more following conditions are TRUE : <i>This is NULL.</i> <i>UdpConfigData.StationAddress</i> neither zero nor one of the configured IP addresses in the underlying IPv6 driver. <i>UdpConfigData.RemoteAddress</i> is not a valid unicast IPv6 address if it is not zero.
EFI_ALREADY_STARTED	The EFI UDPv6 Protocol instance is already started/configured and must be stopped/reset before it can be reconfigured. Only <i>TrafficClass</i> , <i>HopLimit</i> , <i>ReceiveTimeout</i> , and <i>TransmitTimeout</i> can be reconfigured without stopping the current instance of the EFI UDPv6 Protocol.
EFI_ACCESS_DENIED	<i>UdpConfigData.AllowDuplicatePort</i> is FALSE and <i>UdpConfigData.StationPort</i> is already used by other instance.
EFI_OUT_OF_RESOURCES	The EFI UDPv6 Protocol driver cannot allocate memory for this EFI UDPv6 Protocol instance.
EFI_DEVICE_ERROR	An unexpected network or system error occurred and this instance was not opened.

30.2.2.4 EFI_UDP6_PROTOCOL.Groups()

Summary

Joins and leaves multicast groups.

Prototype

```
typedef
EFI_STATUS
(EFI_API *EFI_UDP6_GROUPS) (
    IN EFI_UDP6_PROTOCOL      *This,
    IN BOOLEAN                JoinFlag,
    IN EFI_IPv6_ADDRESS        *MulticastAddress OPTIONAL
);
```

Parameters

This

Pointer to the *EFI_UDP6_PROTOCOL* instance.

JoinFlag

Set to **TRUE** to join a multicast group. Set to **FALSE** to leave one or all multicast groups.

MulticastAddress

Pointer to multicast group address to join or leave.

Description

The *Groups ()* function is used to join or leave one or more multicast group.

If the *JoinFlag* is **FALSE** and the *MulticastAddress* is **NULL**, then all currently joined groups are left.

Status Codes Returned

EFI_SUCCESS	The operation completed successfully.
EFI_NOT_STARTED	The EFI UDPv6 Protocol instance has not been started.
EFI_OUT_OF_RESOURCES	Could not allocate resources to join the group.
EFI_INVALID_PARAMETER	One or more of the following conditions is TRUE : <i>This</i> is NULL . <i>JoinFlag</i> is TRUE and <i>MulticastAddress</i> is *NULL* . <i>JoinFlag</i> is TRUE and <i>**MulticastAddress*</i> is not a valid multicast address.
EFI_ALREADY_STARTED	The group address is already in the group table (when <i>JoinFlag</i> is TRUE).
EFI_NOT_FOUND	The group address is not in the group table (when <i>JoinFlag</i> is FALSE).
EFI_DEVICE_ERROR	An unexpected system or network error occurred.

30.2.2.5 EFI_UDP6_PROTOCOL.Transmit()

Summary

Queues outgoing data packets into the transmit queue.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_UDP6_TRANSMIT) (
    IN EFI_UDP6_PROTOCOL      *This,
    IN EFI_UDP6_COMPLETION_TOKEN *Token
);
```

Parameters

This

Pointer to the *EFI_UDP6_PROTOCOL* instance.

Token

Pointer to the completion token that will be placed into the transmit queue. Type *EFI_UDP6_COMPLETION_TOKEN* is defined in “Related Definitions” below.

Description

The *Transmit()* function places a sending request to this instance of the EFI UDPv6 Protocol, alongside the transmit data that was filled by the user. Whenever the packet in the token is sent out or some errors occur, the *Token.Event* will be signaled and *Token.Status* is updated. Providing a proper notification function and context for the event will enable the user to receive the notification and transmitting status.

Related Definitions

```
//*****
// EFI_UDP6_COMPLETION_TOKEN
//*****
typedef struct {
    EFI_EVENT          Event;
    EFI_STATUS         Status;
    union {
```

(continues on next page)

(continued from previous page)

```
EFI_UDP6_RECEIVE_DATA    *RxData;
EFI_UDP6_TRANSMIT_DATA   *TxData;
}      Packet;
}  EFI_UDP6_COMPLETION_TOKEN;
```

Event

This *Event* will be signaled after the *Status* field is updated by the EFI UDPv6 Protocol driver. The type of *Event* must be *EVT_NOTIFY_SIGNAL*.

Status

Will be set to one of the following values:

EFI_SUCCESS : The receive or transmit operation completed successfully.

EFI_ABORTED : The receive or transmit was aborted.

EFI_TIMEOUT : The transmit timeout expired.

EFI_NETWORK_UNREACHABLE : The destination network is unreachable. *RxData* is set to **NULL** in this situation.

EFI_HOST_UNREACHABLE : The destination host is unreachable. *RxData* is set to **NULL** in this situation.

EFI_PROTOCOL_UNREACHABLE : The UDP protocol is unsupported in the remote system. *RxData* is set to **NULL** in this situation.

EFI_PORT_UNREACHABLE : No service is listening on the remote port. *RxData* is set to **NULL** in this situation.

EFI_ICMP_ERROR : Some other Internet Control Message Protocol (ICMP) error report was received. For example, packets are being sent too fast for the destination to receive them and the destination sent an ICMP source quench report. *RxData* is set to **NULL** in this situation.

EFI_DEVICE_ERROR : An unexpected system or network error occurred.

EFI_SECURITY_VIOLATION : The transmit or receive was failed because of IPsec policy check.

RxData

When this token is used for receiving, *RxData* is a pointer to *EFI_UDP6_RECEIVE_DATA*. Type *EFI_UDP6_RECEIVE_DATA* is defined below.

TxData

When this token is used for transmitting, *TxData* is a pointer to *EFI_UDP6_TRANSMIT_DATA*. Type *EFI_UDP6_TRANSMIT_DATA* is defined below.

The *EFI_UDP6_COMPLETION_TOKEN* structures are used for both transmit and receive operations.

When used for transmitting, the *Event* and *TxData* fields must be filled in by the EFI UDPv6 Protocol client. After the transmit operation completes, the *Status* field is updated by the EFI UDPv6 Protocol and the *Event* is signaled.

When used for receiving, only the *Event* field must be filled in by the EFI UDPv6 Protocol client. After a packet is received, *RxData* and *Status* are filled in by the EFI UDPv6 Protocol and the *Event* is signaled.

```
//*****
// EFI_UDP6_RECEIVE_DATA
//*****
typedef struct {
    EFI_TIME           TimeStamp;
    EFI_EVENT          RecycleSignal;
    EFI_UDP6_SESSION_DATA  UdpSession;
```

(continues on next page)

(continued from previous page)

```

UINT32          DataLength;
UINT32          FragmentCount;
EFI_UDP6_FRAGMENT_DATA  FragmentTable [1];
}  EFI_UDP6_RECEIVE_DATA;

```

TimeStamp

Time when the EFI UDPv6 Protocol accepted the packet. *TimeStamp* is zero filled if timestamps are disabled or unsupported.

RecycleSignal

Indicates the event to signal when the received data has been processed.

UdpSession

The UDP session data including *SourceAddress*, *SourcePort*, *DestinationAddress*, and *DestinationPort*. Type *EFI_UDP6_SESSION_DATA* is defined below.

DataLength

The sum of the fragment data length.

FragmentCount

Number of fragments. Maybe zero.

FragmentTable

Array of fragment descriptors. Maybe zero. Type *EFI_UDP6_FRAGMENT_DATA* is defined below.

EFI_UDP6_RECEIVE_DATA is filled by the EFI UDPv6 Protocol driver when this EFI UDPv6 Protocol instance receives an incoming packet. If there is a waiting token for incoming packets, the *CompletionToken.Packet.RxData* field is updated to this incoming packet and the *CompletionToken.Event* is signaled. The EFI UDPv6 Protocol client must signal the *RecycleSignal* after processing the packet.

FragmentTable could contain multiple buffers that are not in the continuous memory locations. The EFI UDPv6 Protocol client might need to combine two or more buffers in *FragmentTable* to form their own protocol header.

```

//*****
//  EFI_UDP6_SESSION_DATA
//*****
typedef struct {
    EFI_IPv6_ADDRESS    SourceAddress;
    UINT16              SourcePort;
    EFI_IPv6_ADDRESS    DestinationAddress;
    UINT16              DestinationPort;
}  EFI_UDP6_SESSION_DATA;

```

SourceAddress

Address from which this packet is sent. This field should not be used when sending packets.

SourcePort

Port from which this packet is sent. It is in host byte order. This field should not be used when sending packets.

DestinationAddress

Address to which this packet is sent. When sending packet, it'll be ignored if it is zero.

DestinationPort

Port to which this packet is sent. When sending packet, it'll be ignored if it is zero.

The *EFI_UDP6_SESSION_DATA* is used to retrieve the settings when receiving packets or to override the existing settings (only *DestinationAddress* and *DestinationPort* can be overridden) of this EFI UDPv6 Protocol instance when sending packets.


```

//*****
//  EFI_UDP6_FRAGMENT_DATA
//*****
typedef struct {
    UINT32      FragmentLength;
    VOID        *FragmentBuffer;
}  EFI_UDP6_FRAGMENT_DATA;

```

FragmentLength

Length of the fragment data buffer.

FragmentBuffer

Pointer to the fragment data buffer.

EFI_UDP6_FRAGMENT_DATA allows multiple receive or transmit buffers to be specified. The purpose of this structure is to avoid copying the same packet multiple times.

```

//*****
//  EFI_UDP6_TRANSMIT_DATA
//*****
typedef struct {
    EFI_UDP6_SESSION_DATA    *UdpSessionData;
    UINT32                   DataLength;
    UINT32                   FragmentCount;
    EFI_UDP6_FRAGMENT_DATA    FragmentTable [1];
}  EFI_UDP6_TRANSMIT_DATA;

```

UdpSessionData If not **NULL**, the data that is used to override the transmitting settings. Only the two filed *UdpSessionData.DestinationAddress* and *UdpSessionData.DestionPort* can be used as the transmitting setting filed. Type *EFI_UDP6_SESSION_DATA* is defined above.

DataLength

Sum of the fragment data length. Must not exceed the maximum UDP packet size.

FragmentCount

Number of fragments.

FragmentTable

Array of fragment descriptors. Type *EFI_UDP6_FRAGMENT_DATA* is defined above.

The EFI UDPv6 Protocol client must fill this data structure before sending a packet. The packet may contain multiple buffers that may be not in a continuous memory location.

Status Codes Returned

EFI_SUCCESS	The data has been queued for transmission.
EFI_NOT_STARTED	This EFI UDPv6 Protocol instance has not been started.
EFI_NO_MAPPING	The underlying IPv6 driver was responsible for choosing a source address for this instance, but no source address was available for use.

continues on next page

Table 30.12 – continued from previous page

EFI_INVALID_PARAMETER	One or more of the following are TRUE: <i>This</i> is NULL. <i>Token</i> is NULL. <i>Token.Event</i> is NULL. <i>Token.Packet.TxData</i> is NULL. <i>Token.Packet.TxData.FragmentCount</i> is zero. <i>Token.Packet.TxData.DataLength</i> is not equal to the sum of fragment lengths. One or more of the <i>Token.Packet.TxData</i> <i>a.FragmentTable[]</i>.<i>FragmentLength</i> fields is zero. One or more of the <i>Token.Packet.TxData</i> <i>a.FragmentTable[]</i>.<i>FragmentBuffer</i> fields is NULL. <i>Token.Packet.TxData.UdpSessionData</i>. <i>DestinationAddress</i> is not zero and is not valid unicast Ipv6 address if <i>UdpSessionData</i> is not NULL. <i>Token.Packet.TxData.UdpSessionData</i> is NULL and this instance's <i>UdpConfigData</i>. <i>RemoteAddress</i> is unspecified. <i>Token.Packet.TxData</i>. <i>UdpSessionData</i>.<i>DestinationAddress</i> is non-zero when <i>DestinationAddress</i> is configured as non-zero when doing <i>Configure()</i> for this EFI Udp6 protocol instance. <i>Token.Packet.TxData.UdpSessionData</i>.<i>DestinationAddress</i> is zero when <i>DestinationAddress</i> is unspecified when doing <i>Configure()</i> for this EFI Udp6 protocol instance
EFI_ACCESS_DENIED	The transmit completion token with the same <i>Token.Event</i> was already in the transmit queue.
EFI_NOT_READY	The completion token could not be queued because the transmit queue is full.
EFI_OUT_OF_RESOURCES	Could not queue the transmit data.
EFI_NOT_FOUND	There is no route to the destination network or address.
EFI_BAD_BUFFER_SIZE	The data length is greater than the maximum UDP packet size.
EFI_NO_MEDIA	There was a media error.

30.2.2.6 EFI_UDP6_PROTOCOL.Receive()

Summary

Places an asynchronous receive request into the receiving queue.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_UDP6_RECEIVE) (
    IN EFI_UDP6_PROTOCOL      *This,
    IN EFI_UDP6_COMPLETION_TOKEN *Token*
);
```

Parameters

This

Pointer to the *EFI_UDP6_PROTOCOL* instance.

Token

Pointer to a token that is associated with the receive data descriptor. Type *EFI_UDP6_COMPLETION_TOKEN* is defined in *EFI_UDP6_PROTOCOL.Transmit()*.

Description

The *Receive()* function places a completion token into the receive packet queue. This function is always asynchronous.

The caller must fill in the *Token.Event* field in the completion token, and this field cannot be **NULL**. When the receive operation completes, the EFI UDPv6 Protocol driver updates the *Token.Status* and *Token.Packet.RxData* fields and the *Token.Event* is signaled. Providing a proper notification function and context for the event will enable the user to receive the notification and receiving status. That notification function is guaranteed to not be re-entered.

Status Codes Returned

EFI_SUCCESS	The receive completion token was cached.
EFI_NOT_STARTED	This EFI UDPv6 Protocol instance has not been started.
EFI_NO_MAPPING	The underlying IPv6 driver was responsible for choosing a source address for this instance, but no source address was available for use.
EFI_INVALID_PARAMETER	One or more of the following conditions is TRUE : <i>This</i> is NULL . <i>Token</i> is NULL . <i>Token.Event</i> is NULL .
EFI_OUT_OF_RESOURCES	The receive completion token could not be queued due to a lack of system resources (usually memory).
EFI_DEVICE_ERROR	An unexpected system or network error occurred. The EFI UDPv6 Protocol instance has been reset to startup defaults.
EFI_ACCESS_DENIED	A receive completion token with the same <i>Token.Event</i> was already in the receive queue.
EFI_NOT_READY	The receive request could not be queued because the receive queue is full.
EFI_NO_MEDIA	There was a media error.

30.2.2.7 EFI_UDP6_PROTOCOL.Cancel()**Summary**

Aborts an asynchronous transmit or receive request.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_UDP6_CANCEL)(
    IN EFI_UDP6_PROTOCOL          *This,
    IN EFI_UDP6_COMPLETION_TOKEN *Token OPTIONAL
);
```

Parameters

This

Pointer to the *EFI_UDP6_PROTOCOL* instance.

Token

Pointer to a token that has been issued by *EFI_UDP6_PROTOCOL.Transmit()* or *EFI_UDP6_PROTOCOL.Receive()*. If **NULL**, all pending tokens are aborted. Type *EFI_UDP6_COMPLETION_TOKEN* is defined in *EFI_UDP6_PROTOCOL.Transmit()*.

Description

The *Cancel()* function is used to abort a pending transmit or receive request. If the token is in the transmit or receive request queues, after calling this function, *Token.Status* will be set to *EFI_ABORTED* and then *Token.Event* will be signaled. If the token is not in one of the queues, which usually means that the asynchronous operation has completed, this function will not signal the token and *EFI_NOT_FOUND* is returned.

Status Codes Returned

EFI_SUCCESS	The asynchronous I/O request was aborted and <i>Token.Event</i> was signaled. When <i>Token</i> is NULL , all pending requests are aborted and their events are signaled.
EFI_INVALID_PARAMETER	This is NULL .
EFI_NOT_STARTED	This instance has not been started.
EFI_NOT_FOUND	When <i>Token</i> is not NULL , the asynchronous I/O request was not found in the transmit or receive queue. It has either completed or was not issued by <i>Transmit()</i> and <i>Receive()</i> .

30.2.2.8 EFI_UDP6_PROTOCOL.Poll()

Summary

Polls for incoming data packets and processes outgoing data packets.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_UDP6_POLL) (
    IN EFI_UDP6_PROTOCOL    *This
);
```

Parameters

This

Pointer to the *EFI_UDP6_PROTOCOL* instance.

Description

The *Poll()* function can be used by network drivers and applications to increase the rate that data packets are moved between the communications device and the transmit and receive queues.

In some systems, the periodic timer event in the managed network driver may not poll the underlying communications device fast enough to transmit and/or receive all data packets without missing incoming packets or dropping outgoing packets. Drivers and applications that are experiencing packet loss should try calling the *Poll()* function more often.

Status Codes Returned

EFI_SUCCESS	Incoming or outgoing data was processed.
EFI_INVALID_PARAMETER	<i>This is NULL.</i>
EFI_DEVICE_ERROR	An unexpected system or network error occurred.
EFI_TIMEOUT	Data was dropped out of the transmit and/or receive queue. Consider increasing the polling rate.

30.3 EFI MTFTPv4 Protocol

The following sections defines the EFI MTFTPv4 Protocol interface that is built upon the EFI UDPv4 Protocol.

30.3.1 EFI_MTFTP4_SERVICE_BINDING_PROTOCOL

Summary

The EFI MTFTPv4 Service Binding Protocol is used to locate communication devices that are supported by an EFI MTFTPv4 Protocol driver and to create and destroy instances of the EFI MTFTPv4 Protocol child protocol driver that can use the underlying communications device.

GUID

```
#define EFI_MTFTP4_SERVICE_BINDING_PROTOCOL_GUID \
    {0x2e800be, 0x8f01, 0x4aa6, \
     {0x94, 0x6b, 0xd7, 0x13, 0x88, 0xe1, 0x83, 0x3f}}
```

Description

A network application or driver that requires MTFTPv4 I/O services can use one of the protocol handler services, such as *BS->LocateHandleBuffer()*, to search for devices that publish an EFI MTFTPv4 Service Binding Protocol GUID. Each device with a published EFI MTFTPv4 Service Binding Protocol GUID supports the EFI MTFTPv4 Protocol service and may be available for use.

After a successful call to the *EFI_MTFTP4_SERVICE_BINDING_PROTOCOL.CreateChild()* function, the newly created child EFI MTFTPv4 Protocol driver instance is in an unconfigured state; it is not ready to transfer data.

Before a network application terminates execution, every successful call to the *EFI_MTFTP4_SERVICE_BINDING_PROTOCOL.CreateChild()* function must be matched with a call to the *EFI_MTFTP4_SERVICE_BINDING_PROTOCOL.DestroyChild()* function.

Each instance of the EFI MTFTPv4 Protocol driver can support one file transfer operation at a time. To download two files at the same time, two instances of the EFI MTFTPv4 Protocol driver will need to be created.

30.3.2 EFI_MTFTP4_PROTOCOL

Summary

The EFI MTFTPv4 Protocol provides basic services for client-side unicast and/or multicast TFTP operations.

GUID

```
#define EFI_MTFTP4_PROTOCOL_GUID \
    {0x78247c57, 0x63db, 0x4708, \
     {0x99, 0xc2, 0xa8, 0xb4, 0xa9, 0xa6, 0x1f, 0x6b}}
```

Protocol Interface Structure

```
typedef struct _EFI_MTFTP4_PROTOCOL {
    EFI_MTFTP4_GET_MODE_DATA    GetModeData;
    EFI_MTFTP4_CONFIGURE        Configure;
    EFI_MTFTP4_GET_INFO         GetInfo;
    EFI_MTFTP4_PARSE_OPTIONS    ParseOptions;
    EFI_MTFTP4_READ_FILE        ReadFile;
    EFI_MTFTP4_WRITE_FILE       WriteFile;
    EFI_MTFTP4_READ_DIRECTORY   ReadDirectory;
    EFI_MTFTP4_POLL             Poll;
} EFI_MTFTP4_PROTOCOL;
```

Parameters

GetModeData

Reads the current operational settings. See the *GetModeData()* function description.

Configure

Initializes, changes, or resets the operational settings for this instance of the EFI MTFTPv4 Protocol driver. See the *Configure()* function description.

GetInfo

Retrieves information about a file from an MTFTPv4 server. See the *GetInfo()* function description.

ParseOptions

Parses the options in an MTFTPv4 OACK (options acknowledgement) packet. See the *ParseOptions()* function description.

ReadFile

Downloads a file from an MTFTPv4 server. See the *ReadFile()* function description.

WriteFile

Uploads a file to an MTFTPv4 server. This function may be unsupported in some EFI implementations. See the *WriteFile()* function description.

ReadDirectory

Downloads a related file “directory” from an MTFTPv4 server. This function may be unsupported in some EFI implementations. See the *ReadDirectory()* function description.

Poll

Polls for incoming data packets and processes outgoing data packets. See the *Poll()* function description.

Description

The *EFI_MTFTP4_PROTOCOL* is designed to be used by UEFI drivers and applications to transmit and receive data files. The EFI MTFTPv4 Protocol driver uses the underlying EFI UDPv4 Protocol driver and EFI IPv4 Protocol driver.

30.3.3 EFI_MTFTP4_PROTOCOL.GetModeData()

Summary

Reads the current operational settings.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_MTFTP4_GET_MODE_DATA)(
    IN EFI_MTFTP4_PROTOCOL          *This,
```

(continues on next page)

(continued from previous page)

```
OUT EFI_MTFTP4_MODE_DATA    *ModeData
);
```

Parameters

This

Pointer to the *EFI_MTFTP4_PROTOCOL* instance.

ModeData

Pointer to storage for the EFI MTFTPv4 Protocol driver mode data. Type *EFI_MTFTP4_MODE_DATA* is defined in “Related Definitions” below.

Description

The *GetModeData()* function reads the current operational settings of this EFI MTFTPv4 Protocol driver instance.

Related Definitions

```
/**
 * EFI_MTFTP4_MODE_DATA
 */
typedef struct {
    EFI_MTFTP4_CONFIG_DATA    ConfigData;
    UINT8                     SupportedOptionCount;
    UINT8                     **SupportedOptions;
    UINT8                     UnsupportedOptionCount;
    UINT8                     **UnsupportedOptions;
} EFI_MTFTP4_MODE_DATA;
```

ConfigData

The configuration data of this instance. Type *EFI_MTFTP4_CONFIG_DATA* is defined below.

SupportedOptionCount

The number of option strings in the following *SupportedOptions* array.

SupportedOptions

An array of pointers to null-terminated ASCII option strings that are recognized and supported by this EFI MTFTPv4 Protocol driver implementation.

UnsupportedOptionCount

An array of pointers to null-terminated ASCII option strings that are recognized but not supported by this EFI MTFTPv4 Protocol driver implementation.

UnsupportedOptions

An array of option strings that are recognized but are not supported by this EFI MTFTPv4 Protocol driver implementation.

The *EFI_MTFTP4_MODE_DATA* structure describes the operational state of this instance.

```
/**
 * EFI_MTFTP4_CONFIG_DATA
 */
typedef struct {
    BOOLEAN                     UseDefaultSetting;
    EFI_IPv4_ADDRESS            StationIp;
    EFI_IPv4_ADDRESS            SubnetMask;
    UINT16                      LocalPort;
```

(continues on next page)

(continued from previous page)

```

EFI_IPv4_ADDRESS    GatewayIp;
EFI_IPv4_ADDRESS    ServerIp;
UINT16              InitialServerPort;
UINT16              TryCount;
UINT16              TimeoutValue;
} EFI_MTFTP4_CONFIG_DATA;

```

UseDefaultSetting

Set to **TRUE** to use the default station address/subnet mask and the default route table information.

StationIp

If *UseDefaultSetting* is **FALSE**, indicates the station address to use.

SubnetMask

If *UseDefaultSetting* is **FALSE**, indicates the subnet mask to use.

LocalPort

Local port number. Set to zero to use the automatically assigned port number.

GatewayIp

If *UseDefaultSetting* is **FALSE**, indicates the gateway IP address to use.

ServerIp

The IP address of the MTFTPv4 server.

InitialServerPort

The initial MTFTPv4 server port number. Request packets are sent to this port. This number is almost always 69 and using zero defaults to 69.

TryCount

The number of times to transmit MTFTPv4 request packets and wait for a response.

TimeoutValue

The number of seconds to wait for a response after sending the MTFTPv4 request packet.

The *EFI_MTFTP4_CONFIG_DATA* structure is used to report and change MTFTPv4 session parameters.

Status Codes Returned

EFI_SUCCESS	The configuration data was successfully returned.
EFI_OUT_OF_RESOURCES	The required mode data could not be allocated.
EFI_INVALID_PARAMETER	<i>This is NULL or ModeData is NULL.</i>

30.3.4 EFI_MTFTP4_PROTOCOL.Configure()**Summary**

Initializes, changes, or resets the default operational setting for this EFI MTFTPv4 Protocol driver instance.

Prototype

```

typedef
EFI_STATUS
(EFIAPI *EFI_MTFTP4_CONFIGURE)(
    IN EFI_MTFTP4_PROTOCOL      *This,
    IN EFI_MTFTP4_CONFIG_DATA   *MtftpConfigData OPTIONAL
);

```


Parameters**This**

Pointer to the *EFI_MTFTP4_PROTOCOL* instance.

MtftpConfigData

Pointer to the configuration data structure. Type *EFI_MTFTP4_CONFIG_DATA* is defined in *EFI_MTFTP4_PROTOCOL*.*GetModeData()*.

Description

The *Configure()* function is used to set and change the configuration data for this EFI MTFTPv4 Protocol driver instance. The configuration data can be reset to startup defaults by calling *Configure()* with *MtftpConfigData* set to **NULL**. Whenever the instance is reset, any pending operation is aborted. By changing the EFI MTFTPv4 Protocol driver instance configuration data, the client can connect to different MTFTPv4 servers. The configuration parameters in *MtftpConfigData* are used as the default parameters in later MTFTPv4 operations and can be overridden in later operations.

Status Codes Returned

EFI_SUCCESS	The EFI MTFTPv4 Protocol driver was configured successfully.
EFI_INVALID_PARAMETER	One or more following conditions are TRUE: • <i>This</i> is NULL. • <i>MtftpConfigData.UseDefaultSetting</i> is FALSE and <i>MtftpConfigData.StationIp</i> is not a valid IPv4 unicast address. • <i>MtftpCofigData.UseDefaultSetting</i> is FALSE and <i>MtftpConfigData.SubnetMask</i> is invalid. • <i>MtftpCofigData.ServerIp</i> is not a valid IPv4 unicast address. • <i>MtftpConfigData.UseDefaultSetting</i> is FALSE and <i>MtftpConfigData.GatewayIp</i> is not a valid IPv4 unicast address or is not in the same subnet with station address.
EFI_ACCESS_DENIED	The EFI configuration could not be changed at this time because there is one MTFTP background operation in progress.
EFI_NO_MAPPING	When using a default address, configuration (DHCP, BOOTP, RARP, etc.) has not finished yet.
EFI_UNSUPPORTED	A configuration protocol (DHCP, BOOTP, RARP, etc.) could not be located when clients choose to use the default address settings.
EFI_OUT_OF_RESOURCES	The EFI MTFTPv4 Protocol driver instance data could not be allocated.
EFI_DEVICE_ERROR	An unexpected system or network error occurred. The EFI MTFTPv4 Protocol driver instance is not configured.

30.3.5 EFI_MTFTP4_PROTOCOL.GetInfo()**Summary**

Gets information about a file from an MTFTPv4 server.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_MTFTP4_GET_INFO)(
```

(continues on next page)

(continued from previous page)

```

IN EFI_MTFTP4_PROTOCOL      *This,
IN EFI_MTFTP4_OVERRIDE_DATA *OverrideData OPTIONAL,
IN UINT8                   *Filename,
IN UINT8                   *ModeStr OPTIONAL,
IN UINT8                   OptionCount,
IN EFI_MTFTP4_OPTION        *OptionList OPTIONAL,
OUT UINT32                  *PacketLength,
OUT EFI_MTFTP4_PACKET       **Packet OPTIONAL
);

```

Parameters

This

Pointer to the *EFI_MTFTP4_PROTOCOL* instance.

OverrideData

Data that is used to override the existing parameters. If **NULL**, the default parameters that were set in the *EFI_MTFTP4_PROTOCOL* .*Configure()* function are used. Type *EFI_MTFTP4_OVERRIDE_DATA* is defined in “Related Definitions” below.

Filename

Pointer to a null-terminated ASCII file name string.

ModeStr

Pointer to a null-terminated ASCII mode string. If **NULL**, “octet” will be used.

OptionCount

Number of option/value string pairs in *OptionList*.

OptionList

Pointer to array of option/value string pairs. Ignored if *OptionCount* is zero. Type *EFI_MTFTP4_OPTION* is defined in “Related Definitions” below.

PacketLength

The number of bytes in the returned packet.

Packet

The pointer to the received packet. This buffer must be freed by the caller. Type *EFI_MTFTP4_PACKET* is defined in “Related Definitions” below.

Description

The *GetInfo()* function assembles an MTFTPv4 request packet with options; sends it to the MTFTPv4 server; and may return an MTFTPv4 OACK, MTFTPv4 ERROR, or ICMP ERROR packet. Retries occur only if no response packets are received from the MTFTPv4 server before the timeout expires.

Related Definitions

```

//*****
//  EFI_MTFTP_OVERRIDE_DATA
//*****
typedef struct {
    EFI_IPv4_ADDRESS    GatewayIp;
    EFI_IPv4_ADDRESS    ServerIp;
    UINT16              ServerPort;
    UINT16              TryCount;
    UINT16              TimeoutValue;
}  EFI_MTFTP4_OVERRIDE_DATA;

```

GatewayIp

IP address of the gateway. If set to 0.0.0.0, the default gateway address that was set by the *EFI_MTFTP4_PROTOCOL.Configure()* function will not be overridden.

ServerIp

IP address of the MTFTPv4 server. If set to 0.0.0.0, it will use the value that was set by the *EFI_MTFTP4_PROTOCOL.Configure()* function.

ServerPort

MTFTPv4 server port number. If set to zero, it will use the value that was set by the *EFI_MTFTP4_PROTOCOL.Configure()* function.

TryCount

Number of times to transmit MTFTPv4 request packets and wait for a response. If set to zero, it will use the value that was set by the *EFI_MTFTP4_PROTOCOL.Configure()* function.

TimeoutValue

Number of seconds to wait for a response after sending the MTFTPv4 request packet. If set to zero, it will use the value that was set by the *EFI_MTFTP4_PROTOCOL.Configure()* function.

The *EFI_MTFTP4_OVERRIDE_DATA* structure is used to override the existing parameters that were set by the *EFI_MTFTP4_PROTOCOL.Configure()* function.

```
//*****
// EFI_MTFTP4_OPTION
//*****
typedef struct {
    UINT8      *OptionStr;
    UINT8      *ValueStr;
} EFI_MTFTP4_OPTION;
```

OptionStr

Pointer to the null-terminated ASCII MTFTPv4 option string.

ValueStr

Pointer to the null-terminated ASCII MTFTPv4 value string.

```
#pragma pack(1)

//*****
// EFI_MTFTP4_PACKET
//*****
typedef union {
    UINT16      OpCode;
    EFI_MTFTP4_REQ_HEADER  Rrq, Wrq;
    EFI_MTFTP4_OACK_HEADER  Oack;
    EFI_MTFTP4_DATA_HEADER  Data;
    EFI_MTFTP4_ACK_HEADER   Ack;
    // This field should be ignored and treated as reserved
    EFI_MTFTP4_DATA8_HEADER  Data8;
    // This field should be ignored and treated as reserved
    EFI_MTFTP4_ACK8_HEADER   Ack8;
    EFI_MTFTP4_ERROR_HEADER  Error;
} EFI_MTFTP4_PACKET;
```

```

//*****
// EFI_MTF4P4_REQ_HEADER
//*****
typedef struct {
    UINT16          OpCode;
    UINT8           Filename[1];
} EFI_MTF4P4_REQ_HEADER;

//*****
// EFI_MTF4P4_OACK_HEADER
//*****
typedef struct {
    UINT16          OpCode;
    UINT8           Data[1];
} EFI_MTF4P4_OACK_HEADER;

//*****
// EFI_MTF4P4_DATA_HEADER
//*****
typedef struct {
    UINT16          OpCode;
    UINT16          Block;
    UINT8           Data[1];
} EFI_MTF4P4_DATA_HEADER;

//*****
// EFI_MTF4P4_ACK_HEADER
//*****
typedef struct {
    UINT16          OpCode;
    UINT16          Block[1];
} EFI_MTF4P4_ACK_HEADER;

//*****
// EFI_MTF4P4_DATA8_HEADER
// This field should be ignored and treated as reserved
//*****
typedef struct {
    UINT16          OpCode;
    UINT64          Block;
    UINT8           Data[1];
} EFI_MTF4P4_DATA8_HEADER;

```

```

//*****
// EFI_MTF4P4_ACK8_HEADER
// This field should be ignored and treated as reserved
//*****
typedef struct {
    UINT16          OpCode;
    UINT64          Block[1];
} EFI_MTF4P4_ACK8_HEADER;

```

(continues on next page)

(continued from previous page)

```

//*****
// EFI_MTFTP4_ERROR_HEADER
//*****
typedef struct {
    UINT16          OpCode;
    UINT16          ErrorCode;
    UINT8           ErrorMessage[1];
} EFI_MTFTP4_ERROR_HEADER;

#pragma pack()

```

Table, below, *Descriptions of Parameters in MTFTPv4 PacketStructures* describes the parameters that are listed in the MTFTPv4 packet structure definitions above. All the above structures are byte packed. The pragmas may vary from compiler to compiler. The MTFTPv4 packet structures are also used by the following functions:

- *EFI_MTFTP4_PROTOCOL* .ReadFile()
- *EFI_MTFTP4_PROTOCOL* .WriteFile()
- *EFI_MTFTP4_PROTOCOL* .ReadDirectory()
- The EFI MTFTPv4 Protocol packet check callback functions

NOTE: Both incoming and outgoing MTFTPv4 packets are in network byte order. All other parameters defined in functions or data structures are stored in host byte order.

Table 30.18: Descriptions of Parameters in MTFTPv4 PacketStructures

Data Structure	Parameter	Description
<i>EFI_MTFTP4_PACKET</i>	OpCode	Type of packets as defined by the MTFTPv4 packet op-codes. Opcode values are defined below.
	Rrq, Wrq	Read request or write request packet header. See the description for <i>EFI_MTFTP4_REQ_HEADER</i> below in this table
	Oack	Option acknowledge packet header. See the description for <i>EFI_MTFTP4_OACK_HEADER</i> below in this table.
	Data	Data packet header. See the description for <i>EFI_MTFTP4_DATA_HEADER</i> below in this table.
	Ack	Acknowledgement packet header. See the description for <i>EFI_MTFTP4_ACK_HEADER</i> below in this table.
	Data8	This field should be ignored and treated as reserved. Data packet header with big block number. See the description for <i>EFI_MTFTP4_DATA8_HEADER</i> below in this table.

continues on next page

Table 30.18 – continued from previous page

Ack8		
This field should be ignored and treated as reserved.		
Acknowledgement header with big block number. See the description for EFI_MTFFTP4_ACK8_HEADER below in this table.		
Error		
Error packet header. See the description for EFI_MTFFTP4_ERROR_HEADER below in this table.		
EFI_MTFFTP4_REQ_HEADER	OpCode	For this packet type, OpCode = EFI_MTFFTP4_OPCODE_RRQ for a read request or OpCode = EFI_MTFFTP4_OPCODE_WRQ for a write request.
Filename		
The file name to be downloaded or uploaded.		
EFI_MTFFTP4_OACK_HEADER	OpCode	For this packet type, OpCode = EFI_MTFFTP4_OPCODE_OACK.
Data		
The option strings in the option acknowledgement packet.		
EFI_MTFFTP4_DATA_HEADER	OpCode	For this packet type, OpCode = EFI_MTFFTP4_OPCODE_DATA.
Block		
Block number of this data packet.		
Data		
The content of this data packet.		
EFI_MTFFTP4_ACK_HEADER	OpCode	For this packet type, OpCode = EFI_MTFFTP4_OPCODE_ACK.
Block		
The block number of the data packet that is being acknowledged.		
EFI_MTFFTP4_DATA8_HEADER	OpCode	This field should be ignored and treated as reserved. For this packet type, OpCode = EFI_MTFFTP4_OPCODE_DATA8.
Block		
This field should be ignored and treated as reserved.		
The block number of data packet.		
Data		
This field should be ignored and treated as reserved.		
The content of this data packet.		

continues on next page

Table 30.18 – continued from previous page

EFI_MTFTP4_ACK8_HEADER	OpCode	This field should be ignored and treated as reserved.
		For this packet type, OpCode = EFI_MTFTP4_OPCODE_ACK8.
	Block	This field should be ignored and treated as reserved.
		The block number of the data packet that is being acknowledged.
EFI_MTFTP4_ERROR_HEADER	OpCode	For this packet type, OpCode = EFI_MTFTP4_OPCODE_ERROR.
	ErrorCode	The error number as defined by the MTFTPv4 packet error codes. Values for ErrorCode are defined below.
	ErrorMessage	Error message string.

```
//
// MTFTP Packet OpCodes
//
#define EFI_MTFTP4_OPCODE_RRQ    1
#define EFI_MTFTP4_OPCODE_WRQ    2
#define EFI_MTFTP4_OPCODE_DATA    3
#define EFI_MTFTP4_OPCODE_ACK    4
#define EFI_MTFTP4_OPCODE_ERROR  5
#define EFI_MTFTP4_OPCODE_OACK    6
#define EFI_MTFTP4_OPCODE_DIR    7
//This field should be ignored and treated as reserved.
#define EFI_MTFTP4_OPCODE_DATA8    8
//This field should be ignored and treated as reserved.
#define EFI_MTFTP4_OPCODE_ACK8    9
```

Following is a description of the fields in the above definition.

<i>EFI_MTFTP4_OPCODE_RRQ</i>	The MTFTPv4 packet is a read request.
<i>EFI_MTFTP4_OPCODE_WRQ</i>	The MTFTPv4 packet is a write request.
<i>EFI_MTFTP4_OPCODE_DATA</i>	The MTFTPv4 packet is a data packet.
<i>EFI_MTFTP4_OPCODE_ACK</i>	The MTFTPv4 packet is an acknowledgement packet.
<i>EFI_MTFTP4_OPCODE_ERROR</i>	The MTFTPv4 packet is an error packet.
<i>EFI_MTFTP4_OPCODE_OACK</i>	The MTFTPv4 packet is an option acknowledgement packet.
<i>EFI_MTFTP4_OPCODE_DIR</i>	The MTFTPv4 packet is a directory query packet.
<i>EFI_MTFTP4_OPCODE_DATA8</i>	
	This field should be ignored and treated as reserved.
	The MTFTPv4 packet is a data packet with a big block number.

continues on next page

Table 30.19 – continued from previous page

EFI_MTFTP4_OPCODE_ACK8

This field should be ignored and treated as reserved.

The MTFTPv4 packet is an acknowledgement packet with a big block number.

```
//
// MTFTP ERROR Packet ErrorCodes
//
#define EFI_MTFTP4_ERRORCODE_NOT_DEFINED      0
#define EFI_MTFTP4_ERRORCODE_FILE_NOT_FOUND  1
#define EFI_MTFTP4_ERRORCODE_ACCESS_VIOLATION 2
#define EFI_MTFTP4_ERRORCODE_DISK_FULL       3
#define EFI_MTFTP4_ERRORCODE_ILLEGAL_OPERATION 4
#define EFI_MTFTP4_ERRORCODE_UNKNOWN_TRANSFER_ID 5
#define EFI_MTFTP4_ERRORCODE_FILE_ALREADY_EXISTS 6
#define EFI_MTFTP4_ERRORCODE_NO_SUCH_USER    7
#define EFI_MTFTP4_ERRORCODE_REQUEST_DENIED  8
```

<i>EFI_MTFTP4_ERRORCODE_NOT_DEFINED</i>	The error code is not defined. See the error message in the packet (if any) for details.
<i>EFI_MTFTP4_ERRORCODE_FILE_NOT_FOUND</i>	The file was not found.
<i>EFI_MTFTP4_ERRORCODE_ACCESS_VIOLATION</i>	There was an access violation.
<i>EFI_MTFTP4_ERRORCODE_DISK_FULL</i>	The disk was full or its allocation was exceeded.
<i>EFI_MTFTP4_ERRORCODE_ILLEGAL_OPERATION</i>	The MTFTPv4 operation was illegal.
<i>EFI_MTFTP4_ERRORCODE_UNKNOWN_TRANSFER_ID</i>	The transfer ID is unknown.
<i>EFI_MTFTP4_ERRORCODE_FILE_ALREADY_EXISTS</i>	The file already exists.
<i>EFI_MTFTP4_ERRORCODE_NO_SUCH_USER</i>	There is no such user.
<i>EFI_MTFTP4_ERRORCODE_REQUEST_DENIED</i>	The request has been denied due to option negotiation.

Status Codes Returned

<i>EFI_SUCCESS</i>	An MTFTPv4 OACK packet was received and is in the <i>Packet</i> .
--------------------	---

continues on next page

Table 30.21 – continued from previous page

EFI_INVALID_PARAMETER	One or more of the following conditions is TRUE: • <i>This</i> is NULL. • <i>Filename</i> is NULL. • <i>OptionCount</i> is not zero and <i>OptionList</i> is NULL. • One or more options in <i>OptionList</i> have wrong format. • <i>PacketLength</i> is NULL. • One or more IPv4 addresses in <i>OverrideData</i> are not valid unicast IPv4 addresses if <i>OverrideData</i> is not NULL and the addresses are not set to all zero.
EFI_UNSUPPORTED	• One or more options in the <i>OptionList</i> are in the unsupported list of structure <i>EFI_MTFTP4_MODE_DATA</i>.
EFI_NOT_STARTED	The EFI MTFTPv4 Protocol driver has not been started.
EFI_NO_MAPPING	When using a default address, configuration (DHCP, BOOTP, RARP, etc.) has not finished yet.
EFI_ACCESS_DENIED	The previous operation has not completed yet.
EFI_OUT_OF_RESOURCES	Required system resources could not be allocated.
EFI_TFTP_ERROR	An MTFTPv4 ERROR packet was received and is in the <i>Packet</i> .
EFI_NETWORK_UNREACHABLE	An ICMP network unreachable error packet was received and the <i>Packet</i> is set to NULL .
EFI_HOST_UNREACHABLE	An ICMP host unreachable error packet was received and the <i>Packet</i> is set to NULL .
EFI_PROTOCOL_UNREACHABLE	An ICMP protocol unreachable error packet was received and the <i>Packet</i> is set to NULL .
EFI_PORT_UNREACHABLE	An ICMP port unreachable error packet was received and the <i>Packet</i> is set to NULL .
EFI_ICMP_ERROR	Some other ICMP ERROR packet was received and the <i>Packet</i> is set to NULL .
EFI_PROTOCOL_ERROR	An unexpected MTFTPv4 packet was received and is in the <i>Packet</i> .
EFI_TIMEOUT	No responses were received from the MTFTPv4 server.
EFI_DEVICE_ERROR	An unexpected network error or system error occurred.
EFI_NO_MEDIA	There was a media error.

30.3.6 EFI_MTFTP4_PROTOCOL.ParseOptions()

Summary

Parses the options in an MTFTPv4 OACK packet.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_MTFTP4_PARSE_OPTIONS)(
    IN EFI_MTFTP4_PROTOCOL      *This,
    IN UINT32                   PacketLen,
    IN EFI_MTFTP4_PACKET        *Packet,
```

(continues on next page)

(continued from previous page)

```
OUT UINT32                *OptionCount,
OUT EFI_MTFTP4_OPTION     **OptionList OPTIONAL
);
```

Parameters

This

Pointer to the *EFI_MTFTP4_PROTOCOL* instance.

PacketLen

Length of the OACK packet to be parsed.

Packet

Pointer to the OACK packet to be parsed. Type *EFI_MTFTP4_PACKET* is defined in *EFI_MTFTP4_PROTOCOL*.*GetInfo()*.

OptionCount

Pointer to the number of options in following *OptionList*.

OptionList

Pointer to *EFI_MTFTP4_OPTION* storage. Call the EFI Boot Service *FreePool()* to release the *OptionList* if the options in this *OptionList* are not needed any more. Type *EFI_MTFTP4_OPTION* is defined in *EFI_MTFTP4_PROTOCOL*.*GetInfo()*.

Description

The *ParseOptions()* function parses the option fields in an MTFTPv4 OACK packet and returns the number of options that were found and optionally a list of pointers to the options in the packet.

If one or more of the option fields are not valid, then *EFI_PROTOCOL_ERROR* is returned and ***OptionCount** and ***OptionList** stop at the last valid option.

Status Codes Returned

EFI_SUCCESS	The OACK packet was valid and the <i>OptionCount</i> and <i>OptionList</i> parameters have been updated.
EFI_INVALID_PARAMETER	One or more of the following conditions is TRUE : • <i>PacketLen</i> is 0. • <i>Packet</i> is NULL or <i>Packet</i> is not a valid MTFTPv4 packet. • <i>OptionCount</i> is NULL.
EFI_NOT_FOUND	No options were found in the OACK packet.
EFI_OUT_OF_RESOURCES	Storage for the <i>OptionList</i> array cannot be allocated.
EFI_PROTOCOL_ERROR	One or more of the option fields is invalid.

30.3.7 EFI_MTFTP4_PROTOCOL.ReadFile()

Summary

Downloads a file from an MTFTPv4 server.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_MTFTP4_READ_FILE)(
    IN EFI_MTFTP4_PROTOCOL      *This,
    IN EFI_MTFTP4_TOKEN         *Token
);
```

Parameters

This

Pointer to the *EFI_MTFTP4_PROTOCOL* instance.

Token

Pointer to the token structure to provide the parameters that are used in this operation. Type *EFI_MTFTP4_TOKEN* is defined in “Related Definitions” below.

Description

The *ReadFile()* function is used to initialize and start an MTFTPv4 download process and optionally wait for completion. When the download operation completes, whether successfully or not, the *Token.Status* field is updated by the EFI MTFTPv4 Protocol driver and then *Token.Event* is signaled (if it is not **NULL**).

Data can be downloaded from the MTFTPv4 server into either of the following locations:

- A fixed buffer that is pointed to by *Token.Buffer*
- A download service function that is pointed to by *Token.CheckPacket*

If both *Token.Buffer* and *Token.CheckPacket* are used, then *Token.CheckPacket* will be called first. If the call is successful, the packet will be stored in *Token.Buffer*.

Related Definitions

```
/**
// *****
// EFI_MTFTP4_TOKEN
// *****
typedef struct {
    EFI_STATUS          Status;
    EFI_EVENT           Event;
    EFI_MTFTP4_OVERRIDE_DATA *OverrideData;
    UINT8               *Filename;
    UINT8               *ModeStr;
    UINT32              OptionCount;
    EFI_MTFTP4_OPTION    *OptionList;
    UINT64              BufferSize;
    VOID                *Buffer;
    VOID                *Context;
    EFI_MTFTP4_CHECK_PACKET CheckPacket;
    EFI_MTFTP4_TIMEOUT_CALLBACK TimeoutCallback;
    EFI_MTFTP4_PACKET_NEEDED PacketNeeded;
} EFI_MTFTP4_TOKEN;
```

Status

The status that is returned to the caller at the end of the operation to indicate whether this operation completed successfully. Defined *Status* values are listed below.

Event

The event that will be signaled when the operation completes. If set to **NULL**, the corresponding function will wait until the read or write operation finishes. The type of *Event* must be *EVT_NOTIFY_SIGNAL*. The Task Priority Level (TPL) of *Event* must be lower than or equal to *TPL_CALLBACK*.

OverrideData

If not **NULL**, the data that will be used to override the existing configure data. Type *EFI_MTFTP4_OVERRIDE_DATA* is defined in *EFI_MTFTP4_PROTOCOL*.*GetInfo()*.

Filename

Pointer to the null-terminated ASCII file name string.

ModeStr

Pointer to the null-terminated ASCII mode string. If **NULL**, “octet” is used.

OptionCount

Number of option/value string pairs.

OptionList

Pointer to an array of option/value string pairs. Ignored if *OptionCount* is zero. Both a remote server and this driver implementation should support these options. If one or more options are unrecognized by this implementation, it is sent to the remote server without being changed. Type *EFI_MTFTP4_OPTION* is defined in *EFI_MTFTP4_PROTOCOL*.*GetInfo()*.

BufferSize

On input, the size, in bytes, of *Buffer*. On output, the number of bytes transferred

Buffer

Pointer to the data buffer. Data that is downloaded from the MTFTPv4 server is stored here. Data that is uploaded to the MTFTPv4 server is read from here. Ignored if *BufferSize* is zero.

Context

Pointer to the context that will be used by *CheckPacket*, *TimeoutCallback* and *PacketNeeded*.

CheckPacket

Pointer to the callback function to check the contents of the received packet. Type *EFI_MTFTP4_CHECK_PACKET* is defined below.

TimeoutCallback

Pointer to the function to be called when a timeout occurs. Type *EFI_MTFTP4_TIMEOUT_CALLBACK* is defined below.

PacketNeeded

Pointer to the function to provide the needed packet contents. Only used in *WriteFile()* operation. Type *EFI_MTFTP4_PACKET_NEEDED* is defined below.

The *EFI_MTFTP4_TOKEN* structure is used for both the MTFTPv4 reading and writing operations. The caller uses this structure to pass parameters and indicate the operation context. After the reading or writing operation completes, the EFI MTFTPv4 Protocol driver updates the *Status* parameter and the *Event* is signaled if it is not **NULL**. The following table lists the status codes that are returned in the *Status* parameter.

Status Codes Returned

EFI_SUCCESS	The data file has been transferred successfully.
EFI_OUT_OF_RESOURCES	Required system resources could not be allocated.

continues on next page

Table 30.23 – continued from previous page

EFI_BUFFER_TOO_SMALL	BufferSize is not large enough to hold the downloaded data in downloading process.
EFI_ABORTED	Current operation is aborted by user.
EFI_NETWORK_UNREACHABLE	An ICMP network unreachable error packet was received.
EFI_NETWORK_UNREACHABLE	An ICMP host unreachable error packet was received.
EFI_NETWORK_UNREACHABLE	An ICMP protocol unreachable error packet was received.
EFI_NETWORK_UNREACHABLE	An ICMP port unreachable error packet was received.
EFI_ICMP_ERROR	Some other ICMP ERROR packet was received.
EFI_TIMEOUT	No responses were received from the MTFTPv4 server.
EFI_TFTP_ERROR	An MTFTPv4 ERROR packet was received.
EFI_DEVICE_ERROR	An unexpected network error or system error occurred.
EFI_NO_MEDIA	There was a media error.

```

//*****
// EFI_MTFTP4_CHECK_PACKET
//*****
typedef
EFI_STATUS
(EFI_API *EFI_MTFTP4_CHECK_PACKET) (
    IN EFI_MTFTP4_PROTOCOL      *This,
    IN EFI_MTFTP4_TOKEN         *Token,
    IN UINT16                   PacketLen,
    IN EFI_MTFTP4_PACKET        *Packet
);

```

This

Pointer to the *EFI_MTFTP4_PROTOCOL* instance.

Token

The token that the caller provided in the *EFI_MTFTP4_PROTOCOL.ReadFile()*, *WriteFile()* or *ReadDirectory()* function. Type *EFI_MTFTP4_TOKEN* is defined in *EFI_MTFTP4_PROTOCOL.ReadFile()*.

PacketLen

Indicates the length of the packet.

Packet

Pointer to an MTFTPv4 packet. Type *EFI_MTFTP4_PACKET* is defined in *EFI_MTFTP4_PROTOCOL.GetInfo()*.

EFI_MTFTP4_CHECK_PACKET is a callback function that is provided by the caller to intercept the *EFI_MTFTP4_OPCODE_DATA* or *EFI_MTFTP4_OPCODE_DATA8* packets processed in the *EFI_MTFTP4_PROTOCOL.ReadFile()* function, and alternatively to intercept *EFI_MTFTP4_OPCODE_OACK* or *EFI_MTFTP4_OPCODE_ERROR* packets during a call to *EFI_MTFTP4_PROTOCOL.ReadFile()*, *WriteFile()* or *ReadDirectory()*. Whenever an MTFTPv4 packet with the type described above is received from a server, the EFI MTFTPv4 Protocol driver will call *EFI_MTFTP4_CHECK_PACKET* function to let the caller have an opportunity to process this packet. Any status code other than *EFI_SUCCESS* that is returned from this function will abort the transfer process.

```

//*****
// EFI_MTFTP4_TIMEOUT_CALLBACK

```

(continues on next page)

(continued from previous page)

```

//*****
typedef
EFI_STATUS
(EFIAPI *EFI_MTF4_TIMEOUT_CALLBACK) (
    IN EFI_MTF4_PROTOCOL      *This,
    IN EFI_MTF4_TOKEN         *Token
);

```

This

Pointer to the *EFI_MTF4_PROTOCOL* instance.

Token

The token that is provided in the *EFI_MTF4_PROTOCOL*.*ReadFile()* or *EFI_MTF4_PROTOCOL*.*WriteFile()* or *EFI_MTF4_PROTOCOL*.*ReadDirectory()* functions by the caller. Type *EFI_MTF4_TOKEN* is defined in *EFI_MTF4_PROTOCOL*.*ReadFile()*.

EFI_MTF4_TIMEOUT_CALLBACK is a callback function that the caller provides to capture the timeout event in the *EFI_MTF4_PROTOCOL*.*ReadFile()*, *EFI_MTF4_PROTOCOL*.*WriteFile()* or *EFI_MTF4_PROTOCOL*.*ReadDirectory()* functions. Whenever a timeout occurs, the EFI MTF4 Protocol driver will call the *EFI_MTF4_TIMEOUT_CALLBACK* function to notify the caller of the timeout event. Any status code other than *EFI_SUCCESS* that is returned from this function will abort the current download process.

```

//*****
// EFI_MTF4_PACKET_NEEDED
//*****
typedef
EFI_STATUS
(EFIAPI *EFI_MTF4_PACKET_NEEDED) (
    IN EFI_MTF4_PROTOCOL      *This,
    IN EFI_MTF4_TOKEN         *Token,
    IN OUT UINT16             *Length,
    OUT VOID                  **Buffer
);

```

This

Pointer to the *EFI_MTF4_PROTOCOL* instance.

Token

The token provided in the *EFI_MTF4_PROTOCOL*.*WriteFile()* by the caller.

Length

Indicates the length of the raw data wanted on input, and the length the data available on output.

Buffer

Pointer to the buffer where the data is stored.

EFI_MTF4_PACKET_NEEDED is a callback function that the caller provides to feed data to the *EFI_MTF4_PROTOCOL*.*WriteFile()* function. *EFI_MTF4_PACKET_NEEDED* provides another mechanism for the caller to provide data to upload other than a static buffer. The EFI MTF4 Protocol driver always calls *EFI_MTF4_PACKET_NEEDED* to get packet data from the caller if no static buffer was given in the initial call to *EFI_MTF4_PROTOCOL*.*WriteFile()* function. Setting ***Length** to zero signals the end of the session. Returning a status code other than *EFI_SUCCESS* aborts the session.

Status Codes Returned

EFI_SUCCESS	The data file is being downloaded.
EFI_INVALID_PARAMETER	One or more of the parameters is not valid. • <i>This</i> is NULL. • <i>Token</i> is NULL. • <i>Token.FileName</i> is NULL. • <i>Token.OptionCount</i> is not zero and <i>Token.OptionList</i> is NULL. • One or more options in <i>Token.OptionList</i> have wrong format. • <i>Token.Buffer</i> and <i>Token.CheckPacket</i> are both NULL. • One or more IPv4 addresses in <i>Token.OverrideData</i> are not valid unicast IPv4 addresses if <i>Token.OverrideData</i> is not NULL and the addresses are not set to all zero.
EFI_UNSUPPORTED	• One or more options in the <i>Token.OptionList</i> are in the unsupported list of structure <i>EFI_MTFTP4_MODE_DATA</i>.
EFI_NOT_STARTED	The EFI MTFTPv4 Protocol driver has not been started.
EFI_NO_MAPPING	When using a default address, configuration (DHCP, BOOTP, RARP, etc.) is not finished yet.
EFI_ALREADY_STARTED	This <i>Token</i> is being used in another MTFTPv4 session.
EFI_ACCESS_DENIED	The previous operation has not completed yet.
EFI_OUT_OF_RESOURCES	Required system resources could not be allocated.
EFI_DEVICE_ERROR	An unexpected network error or system error occurred.
EFI_NO_MEDIA	There was a media error.

30.3.8 EFI_MTFTP4_PROTOCOL.WriteFile()

Summary

Sends a data file to an MTFTPv4 server. May be unsupported in some EFI implementations.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_MTFTP4_WRITE_FILE)(
    IN EFI_MTFTP4_PROTOCOL      *This,
    IN EFI_MTFTP4_TOKEN         *Token
);
```

Parameters

This

Pointer to the *EFI_MTFTP4_PROTOCOL* instance.

Token

Pointer to the token structure to provide the parameters that are used in this function. Type *EFI_MTFTP4_TOKEN* is defined in *EFI_MTFTP4_PROTOCOL.ReadFile()*.

Description

The *WriteFile()* function is used to initialize an uploading operation with the given option list and optionally wait for completion. If one or more of the options is not supported by the server, the unsupported options are ignored and a standard TFTP process starts instead. When the upload process completes, whether successfully or not, *Token.Event* is signaled, and the EFI MTFTPv4 Protocol driver updates *Token.Status*.

The caller can supply the data to be uploaded in the following two modes:

- Through the user-provided buffer
- Through a callback function

With the user-provided buffer, the *Token.BufferSize* field indicates the length of the buffer, and the driver will upload the data in the buffer. With an *EFI_MTFTP4_PACKET_NEEDED* callback function, the driver will call this callback function to get more data from the user to upload. See the definition of *EFI_MTFTP4_PACKET_NEEDED* for more information. These two modes cannot be used at the same time. The callback function will be ignored if the user provides the buffer.

Status Codes Returned

EFI_SUCCESS	The upload session has started.
EFI_UNSUPPORTED	The operation is not supported by this implementation.
EFI_INVALID_PARAMETER	One or more of the following conditions is TRUE: • <i>This</i> is NULL. • <i>Token</i> is NULL. • <i>Token.Filename</i> is NULL. • <i>Token.OptionCount</i> is not zero and <i>Token.OptionList</i> is NULL. • One or more options in <i>Token.OptionList</i> have wrong format. • <i>Token.Buffer</i> and <i>Token.PacketNeeded</i> are both NULL. • One or more IPv4 addresses in <i>Token.OverrideData</i> are not valid unicast IPv4 addresses if <i>Token.OverrideData</i> is not NULL and the addresses are not set to all zero.
EFI_UNSUPPORTED	• One or more options in the <i>Token.OptionList</i> are in the unsupported list of structure <i>EFI_MTFTP4_MODE_DATA</i>.
EFI_NOT_STARTED	The EFI MTFTPv4 Protocol driver has not been started.
EFI_NO_MAPPING	When using a default address, configuration (DHCP, BOOTP, RARP, etc.) is not finished yet.
EFI_ALREADY_STARTED	This <i>Token</i> is already being used in another MTFTPv4 session.
EFI_OUT_OF_RESOURCES	Required system resources could not be allocated.
EFI_ACCESS_DENIED	The previous operation has not completed yet.
EFI_DEVICE_ERROR	An unexpected network error or system error occurred.
EFI_NO_MEDIA	There was a media error.

30.3.9 EFI_MTFTP4_PROTOCOL.ReadDirectory()

Summary

Downloads a data file “directory” from an MTFTPv4 server. May be unsupported in some EFI implementations.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_MTFTP4_READ_DIRECTORY) (
    IN EFI_MTFTP4_PROTOCOL      *This,
    IN EFI_MTFTP4_TOKEN         *Token
);
```

Parameters

This

Pointer to the *EFI_MTFTP4_PROTOCOL* instance.

Token

Pointer to the token structure to provide the parameters that are used in this function. Type *EFI_MTFTP4_TOKEN* is defined in *EFI_MTFTP4_PROTOCOL.ReadFile()*.

Description

The *ReadDirectory()* function is used to return a list of files on the MTFTPv4 server that are logically (or operationally) related to *Token.Filename*. The directory request packet that is sent to the server is built with the option list that was provided by caller, if present.

The file information that the server returns is put into either of the following locations:

- A fixed buffer that is pointed to by *Token.Buffer*
- A download service function that is pointed to by *Token.CheckPacket*

If both *Token.Buffer* and *Token.CheckPacket* are used, then *Token.CheckPacket* will be called first. If the call is successful, the packet will be stored in *Token.Buffer*.

The returned directory listing in the *Token.Buffer* or *EFI_MTFTP4_PACKET* consists of a list of two or three variable-length ASCII strings, each terminated by a null character, for each file in the directory. If the multicast option is involved, the first field of each directory entry is the static multicast IP address and UDP port number that is associated with the file name. The format of the field is *ip:ip:ip:ip:port*. If the multicast option is not involved, this field and its terminating null character are not present.

The next field of each directory entry is the file name and the last field is the file information string. The information string contains the file size and the create/modify timestamp. The format of the information string is *filesize yyyy-mm-dd hh:mm:ss:ffff*. The timestamp is Coordinated Universal Time (UTC; also known as Greenwich Mean Time [GMT]).

Status Codes Returned

EFI_SUCCESS	The MTFTPv4 related file “directory” has been downloaded.
EFI_UNSUPPORTED	The EFI MTFTPv4 Protocol driver does not support this function.

continues on next page

Table 30.26 – continued from previous page

EFI_INVALID_PARAMETER	One or more of these conditions is TRUE : • <i>This</i> is NULL. • <i>Token</i> is NULL. • <i>Token.Filename</i> is NULL. • <i>Token.OptionCount</i> is not zero and <i>Token.OptionList</i> is NULL. • One or more options in <i>Token.OptionList</i> have wrong format. <i>Token.Buffer</i> and <i>Token.CheckPacket</i> are both **NULL*. • One or more IPv4 addresses in <i>Token.OverrideData</i> are not valid unicast IPv4 addresses if <i>Token.OverrideData</i> is not NULL and the addresses are not set to all zero.
EFI_UNSUPPORTED	One or more options in the <i>Token.OptionList</i> are in the unsupported list of structure <i>EFI_MTFTP4_MODE_DATA</i> .
EFI_NOT_STARTED	The EFI MTFTPv4 Protocol driver has not been started.
EFI_NO_MAPPING	When using a default address, configuration (DHCP, BOOTP, RARP, etc.) is not finished yet.
EFI_ALREADY_STARTED	This <i>Token</i> is already being used in another MTFTPv4 session.
EFI_OUT_OF_RESOURCES	Required system resources could not be allocated.
EFI_ACCESS_DENIED	The previous operation has not completed yet.
EFI_DEVICE_ERROR	An unexpected network error or system error occurred.
EFI_NO_MEDIA	There was a media error.

30.3.10 EFI_MTFTP4_PROTOCOL.POLL()

Summary

Polls for incoming data packets and processes outgoing data packets.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_MTFTP4_POLL) (
    IN EFI_MTFTP4_PROTOCOL    *This
);
```

Parameters

This

Pointer to the *EFI_MTFTP4_PROTOCOL* instance.

Description

The *Poll()* function can be used by network drivers and applications to increase the rate that data packets are moved between the communications device and the transmit and receive queues.

In some systems, the periodic timer event in the managed network driver may not poll the underlying communications device fast enough to transmit and/or receive all data packets without missing incoming packets or dropping outgoing packets. Drivers and applications that are experiencing packet loss should try calling the *Poll()* function more often.

Status Codes Returned

EFI_SUCCESS	Incoming or outgoing data was processed.
EFI_NOT_STARTED	This EFI MTFTPv4 Protocol instance has not been started.
EFI_NO_MAPPING	When using a default address, configuration (DHCP, BOOTP, RARP, etc.) is not finished yet.
EFI_INVALID_PARAMETER	<i>This is NULL.</i>
EFI_DEVICE_ERROR	An unexpected system or network error occurred.
EFI_TIMEOUT	Data was dropped out of the transmit and/or receive queue. Consider increasing the polling rate.

30.4 EFI MTFTPv6 Protocol

This section defines the EFI MTFTPv6 Protocol interface that is built upon the EFI UDPv6 Protocol.

30.4.1 MTFTP6 Service Binding Protocol

30.4.1.1 EFI_MTFTP6_SERVICE_BINDING_PROTOCOL

Summary

The EFI MTFTPv6 Service Binding Protocol is used to locate communication devices that are supported by an EFI MTFTPv6 Protocol driver and to create and destroy instances of the EFI MTFTPv6 Protocol child instance that can use the underlying communications device.

GUID

```
#define EFI_MTFTP6_SERVICE_BINDING_PROTOCOL_GUID \
    {0xd9760ff3, 0x3cca, 0x4267, \
     {0x80, 0xf9, 0x75, 0x27, 0xfa, 0xfa, 0x42, 0x23}}
```

Description

A network application or driver that requires MTFTPv6 I/O services can use one of the protocol handler services, such as *BS->LocateHandleBuffer()*, to search for devices that publish an EFI MTFTPv6 Service Binding Protocol GUID. Each device with a published EFI MTFTPv6 Service Binding Protocol GUID supports the EFI MTFTPv6 Protocol service and may be available for use.

After a successful call to the *EFI_MTFTP6_SERVICE_BINDING_PROTOCOL.CreateChild()* function, the newly created child EFI MTFTPv6 Protocol driver instance is in the un-configured state; it is not ready to transfer data.

Before a network application terminates execution, every successful call to the *EFI_MTFTP6_SERVICE_BINDING_PROTOCOL.CreateChild()* function must be matched with a call to the *EFI_MTFTP6_SERVICE_BINDING_PROTOCOL.DestroyChild()* function.

Each instance of the EFI MTFTPv6 Protocol driver can support one file transfer operation at a time. To download two files at the same time, two instances of the EFI MTFTPv6 Protocol driver need to be created.

30.4.2 MTFTP6 Protocol

30.4.2.1 EFI_MTFTP6_PROTOCOL

Summary

The EFI MTFTPv6 Protocol provides basic services for client-side unicast and/or multicast TFTP operations.

GUID

```
#define EFI_MTFTP6_PROTOCOL_GUID \
    {0xbf0a78ba, 0xec29, 0x49cf, \
     0xa1, 0xc9, 0x7a, 0xe5, 0x4e, 0xab, 0x6a, 0x51}}
```

Protocol Interface Structure

```
typedef struct _EFI_MTFTP6_PROTOCOL {
    EFI_MTFTP6_GET_MODE_DATA    GetModeData;
    EFI_MTFTP6_CONFIGURE        Configure;
    EFI_MTFTP6_GET_INFO         GetInfo;
    EFI_MTFTP6_PARSE_OPTIONS    ;
    EFI_MTFTP6_READ_FILE        ReadFile;
    EFI_MTFTP6_WRITE_FILE       WriteFile;
    EFI_MTFTP6_READ_DIRECTORY   ReadDirectory;
    EFI_MTFTP6_POLL             Poll;
} EFI_MTFTP6_PROTOCOL;
```

Parameters

GetModeData

Reads the current operational settings. See the *GetModeData()* function description.

Configure

Initializes, changes, or resets the operational settings for this instance of the EFI MTFTPv6 Protocol driver. See the *Configure()* function description.

GetInfo

Retrieves information about a file from an MTFTPv6 server. See the *GetInfo()* function description. Parses the options in an MTFTPv6 OACK (options acknowledgement) packet. See the *()* function description.

ReadFile

Downloads a file from an MTFTPv6 server. See the *ReadFile()* function description.

WriteFile

Uploads a file to an MTFTPv6 server. This function may be unsupported in some EFI implementations. See the *WriteFile()* function description.

ReadDirectory

Downloads a related file directory from an MTFTPv6 server. This function may be unsupported in some EFI implementations. See the *ReadDirectory()* function description.

Poll

Polls for incoming data packets and processes outgoing data packets. See the *Poll()* function description.

Description

The *EFI_MTFTP6_PROTOCOL* is designed to be used by UEFI drivers and applications to transmit and receive data files. The EFI MTFTPv6 Protocol driver uses the underlying EFI UDPv6 Protocol driver and EFI IPv6 Protocol driver.

30.4.2.2 EFI_MTFTP6_PROTOCOL.GetModeData()

Summary

Read the current operational settings.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_MTFTP6_GET_MODE_DATA)(
    IN EFI_MTFTP6_PROTOCOL          *This,
    OUT EFI_MTFTP6_MODE_DATA        *ModeData
);
```

Parameters

This

Pointer to the *EFI_MTFTP6_PROTOCOL* instance.

ModeData

The buffer in which the EFI MTFTPv6 Protocol driver mode data is returned. Type *EFI_MTFTP6_MODE_DATA* is defined in “Related Definitions” below.

Description

The *GetModeData* () function reads the current operational settings of this EFI MTFTPv6 Protocol driver instance.

Related Definitions

```
/**
//*****
//  EFI_MTFTP6_MODE_DATA
//*****
typedef struct {
    EFI_MTFTP6_CONFIG_DATA    ConfigData;
    UINT8                     SupportedOptionCount;
    UINT8                     **SupportedOptions;
} EFI_MTFTP6_MODE_DATA;
```

ConfigData

The configuration data of this instance. Type *EFI_MTFTP6_CONFIG_DATA* is defined below.

SupportedOptionCount

The number of option strings in the following *SupportedOptions* array.

SupportedOptions

An array of null-terminated ASCII option strings that are recognized and supported by this EFI MTFTPv6 Protocol driver implementation. The buffer is read only to the caller and the caller should NOT free the buffer.

The *EFI_MTFTP6_MODE_DATA* structure describes the operational state of this instance.

```
/**
//*****
//  EFI_MTFTP6_CONFIG_DATA
//*****
typedef struct {
    EFI_IPv6_ADDRESS    StationIp;
    UINT16               LocalPort;
    EFI_IPv6_ADDRESS    ServerIp;
    UINT16               InitialServerPort;
```

(continues on next page)

(continued from previous page)

```

UINT16          TryCount;
UINT16          TimeoutValue;
} EFI_MTFTP6_CONFIG_DATA;

```

StationIp

The local IP address to use. Set to zero to let the underlying IPv6 driver choose a source address. If not zero it must be one of the configured IP addresses in the underlying IPv6 driver.

LocalPort

Local port number. Set to zero to use the automatically assigned port number.

ServerIp

The IP address of the MTFTPv6 server.

InitialServerPort

The initial MTFTPv6 server port number. Request packets are sent to this port. This number is almost always 69 and using zero defaults to 69.

TryCount

The number of times to transmit MTFTPv6 request packets and wait for a response.

TimeoutValue

The number of seconds to wait for a response after sending the MTFTPv6 request packet.

The *EFI_MTFTP6_CONFIG_DATA* structure is used to retrieve and change MTFTPv6 session parameters.

Status Codes Returned

EFI_SUCCESS	The configuration data was successfully returned.
EFI_OUT_OF_RESOURCES	The required mode data could not be allocated.
EFI_INVALID_PARAMETER	<i>This is NULL or ModeData is NULL.</i>

30.4.2.3 EFI_MTFTP6_PROTOCOL.Configure()**Summary**

Initializes, changes, or resets the default operational setting for this EFI MTFTPv6 Protocol driver instance.

Prototype

```

typedef
EFI_STATUS
(EFIAPI *EFI_MTFTP6_CONFIGURE)(
    IN EFI_MTFTP6_PROTOCOL      *This,
    IN EFI_MTFTP6_CONFIG_DATA   *MftpConfigData OPTIONAL
);

```

Parameters**This**

Pointer to the *EFI_MTFTP6_PROTOCOL* instance.

MftpConfigData

Pointer to the configuration data structure. Type *EFI_MTFTP6_CONFIG_DATA* is defined in *EFI_MTFTP6_PROTOCOL.GetModeData()*.

Description

The *Configure* () function is used to set and change the configuration data for this EFI MTFTPv6 Protocol driver instance. The configuration data can be reset to startup defaults by calling *Configure* () with *MtftpConfigData* set to NULL. Whenever the instance is reset, any pending operation is aborted. By changing the EFI MTFTPv6 Protocol driver instance configuration data, the client can connect to different MTFTPv6 servers. The configuration parameters in *MtftpConfigData* are used as the default parameters in later MTFTPv6 operations and can be overridden in later operations.

Status Codes Returned

EFI_SUCCESS	The EFI MTFTPv6 Protocol instance was configured successfully.
EFI_INVALID_PARAMETER	One or more following conditions are TRUE: • This is NULL. • <i>MtftpConfigData.StationIp</i> is neither zero nor one of the configured IP addresses in the underlying IPv6 driver. • <i>MtftpCofigData.ServerIp</i> is not a valid IPv6 unicast address.
EFI_ACCESS_DENIED	• The configuration could not be changed at this time because there is some MTFTP background operation in progress. • <i>MtftpCofigData.LocalPort</i> is already in use.
EFI_NO_MAPPING	The underlying IPv6 driver was responsible for choosing a source address for this instance, but no source address was available for use.
EFI_OUT_OF_RESOURCES	The EFI MTFTPv6 Protocol driver instance data could not be allocated.
EFI_DEVICE_ERROR	An unexpected system or network error occurred. The EFI MTFTPv6 Protocol driver instance is not configured.

30.4.2.4 EFI_MTFTP6_PROTOCOL.GetInfo()**Summary**

Get information about a file from an MTFTPv6 server.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_MTFTP6_GET_INFO)(
    IN EFI_MTFTP6_PROTOCOL      *This,
    IN EFI_MTFTP6_OVERRIDE_DATA *OverrideData OPTIONAL,
    IN UINT8                    *Filename,
    IN UINT8                    *ModeStr OPTIONAL,
    IN UINT8                    OptionCount,
    IN EFI_MTFTP6_OPTION        *OptionList OPTIONAL,
    OUT UINT32                   *PacketLength,
    OUT EFI_MTFTP6_PACKET       **Packet OPTIONAL
);
```

Parameters

This

Pointer to the *EFI_MTFTP6_PROTOCOL* instance.

OverrideData

Data that is used to override the existing parameters. If **NULL**, the default parameters that were set in the *EFI_MTFTP6_PROTOCOL.Configure()* function are used. Type *EFI_MTFTP6_OVERRIDE_DATA* is defined in “Related Definitions” below.

Filename

Pointer to a null-terminated ASCII file name string.

ModeStr

Pointer to a null-terminated ASCII mode string. If **NULL**, octet will be used.

OptionCount

Number of option/value string pairs in *OptionList*.

OptionList

Pointer to array of option/value string pairs. Ignored if *OptionCount* is zero. Type *EFI_MTFTP6_OPTION* is defined in “Related Definitions” below.

PacketLength

The number of bytes in the returned packet.

Packet

The pointer to the received packet. This buffer must be freed by the caller. Type *EFI_MTFTP6_PACKET* is defined in “Related Definitions” below.

Description

The *GetInfo()* function assembles an MTFTPv6 request packet with options, sends it to the MTFTPv6 server, and may return an MTFTPv6 OACK, MTFTPv6 ERROR, or ICMP ERROR packet. Retries occur only if no response packets are received from the MTFTPv6 server before the timeout expires.

Related Definitions

```

//*****
// EFI_MTFTP6_OVERRIDE_DATA
//*****
typedef struct {
    EFI_IPv6_ADDRESS    ServerIp;
    UINT16              ServerPort;
    UINT16              TryCount;
    UINT16              TimeoutValue;
} EFI_MTFTP6_OVERRIDE_DATA;

```

ServerIp

IP address of the MTFTPv6 server. If set to all zero, the value that was set by the *EFI_MTFTP6_PROTOCOL.Configure()* function will be used.

ServerPort

MTFTPv6 server port number. If set to zero, it will use the value that was set by the *EFI_MTFTP6_PROTOCOL.Configure()* function.

TryCount

Number of times to transmit MTFTPv6 request packets and wait for a response. If set to zero, the value that was set by the *EFI_MTFTP6_PROTOCOL.Configure()* function will be used.

TimeoutValue

Number of seconds to wait for a response after sending the MTFTPv6 request packet. If set to zero, the value that was set by the *EFI_MTFTP6_PROTOCOL.Configure()* function will be used.

The *EFI_MTF6_OVERRIDE_DATA* structure is used to override the existing parameters that were set by the *EFI_MTF6_PROTOCOL.Configure()* function.

```

//*****
// EFI_MTF6_OPTION
//*****
typedef struct {
    UINT8      *OptionStr;
    UINT8      *ValueStr;
} EFI_MTF6_OPTION;
    
```

OptionStr

Pointer to the null-terminated ASCII MTF6v6 option string.

ValueStr

Pointer to the null-terminated ASCII MTF6v6 value string.

```

#pragma pack(1)

//*****
// EFI_MTF6_PACKET
//*****
typedef union {
    UINT16      OpCode;
    EFI_MTF6_REQ_HEADER  Rrq;
    EFI_MTF6_REQ_HEADER  Wrq;
    EFI_MTF6_OACK_HEADER  Oack;
    EFI_MTF6_DATA_HEADER  Data;
    EFI_MTF6_ACK_HEADER   Ack;
    // This field should be ignored and treated as reserved.
    EFI_MTF6_DATA8_HEADER Data8;
    // This field should be ignored and treated as reserved.
    EFI_MTF6_ACK8_HEADER  Ack8;
    EFI_MTF6_ERROR_HEADER Error;
} EFI_MTF6_PACKET;

//*****
// EFI_MTF6_REQ_HEADER
//*****
typedef struct {
    UINT16      OpCode;
    UINT8      Filename[1];
} EFI_MTF6_REQ_HEADER;
    
```

```

//*****
// EFI_MTF6_OACK_HEADER
//*****
typedef struct {
    UINT16      OpCode;
    UINT8      Data[1];
} EFI_MTF6_OACK_HEADER;

//*****
// EFI_MTF6_DATA_HEADER
    
```

(continues on next page)

(continued from previous page)

```

//*****
typedef struct {
    UINT16      OpCode;
    UINT16      Block;
    UINT8       Data[1];
}  EFI_MTFTP6_DATA_HEADER;

//*****
// EFI_MTFTP6_ACK_HEADER
//*****
typedef struct {
    UINT16      OpCode;
    UINT16      Block[1];
}  EFI_MTFTP6_ACK_HEADER;

//*****
// EFI_MTFTP6_DATA8_HEADER
// This field should be ignored and treated as reserved.
//*****
typedef struct {
    UINT16      OpCode;
    UINT64      Block;
    UINT8       Data[1];
}  EFI_MTFTP6_DATA8_HEADER;

```

```

//*****
// EFI_MTFTP6_ACK8_HEADER
//*****
typedef struct {
    UINT16      OpCode;
    UINT64      Block[1];
}  EFI_MTFTP6_ACK8_HEADER;

//*****
// EFI_MTFTP6_ERROR_HEADER
//*****
typedef struct {
    UINT16      OpCode;
    UINT16      ErrorCode;
    UINT8       ErrorMessage[1];
}  EFI_MTFTP6_ERROR_HEADER;

#pragma pack()

```

Table 1 below describes the parameters that are listed in the MTFTPv6 packet structure definitions above. All the above structures are byte packed. The pragmas may vary from compiler to compiler. The MTFTPv6 packet structures are also used by the following functions:

- *EFI_MTFTP6_PROTOCOL.ReadFile()*
- *EFI_MTFTP6_PROTOCOL.WriteFile()*
- *EFI_MTFTP6_PROTOCOL.ReadDirectory()*

- The EFI MTFTPv6 Protocol packet check callback functions

NOTE: BYTE ORDER: Both incoming and outgoing MTFTPv6 packets are in network byte order. All other parameters defined in functions or data structures are stored in host byte order.

Table 30.30: Descriptions of Parameters in MTFTPv6 Packet Structures

Data Structure	Parameter	Description
EFI_MTFTP6_PACKET	OpCode	Type of packets as defined by the MTFTPv6 packet op-codes. Opcode values are defined below.
	Rrq, Wrq	Read request or write request packet header. See the description for <i>EFI_MTFTP6_REQ_HEADER</i> below in this table.
	Oack	Option acknowledge packet header. See the description for <i>EFI_MTFTP6_OACK_HEADER</i> below in this table.
	Data	Data packet header. See the description for <i>EFI_MTFTP6_DATA_HEADER</i> below in this table.
	Ack	Acknowledgement packet header. See the description for <i>EFI_MTFTP6_ACK_HEADER</i> below in this table.
	Data8	This field should be ignored and treated as reserved. Data packet header with big block number. See the description for <i>EFI_MTFTP6_DATA8_HEADER</i> below in this table.
	Ack8	This field should be ignored and treated as reserved. Acknowledgement header with big block number. See the description for <i>EFI_MTFTP6_ACK8_HEADER</i> below in this table.
	Error	Error packet header. See the description for <i>EFI_MTFTP6_ERROR_HEADER</i> below in this table.
<i>EFI_MTFTP6_REQ_HEADER</i>	OpCode	For this packet type, <i>OpCode</i> = <i>EFI_MTFTP6_OPCODE_RRQ</i> for a read request or <i>OpCode</i> = <i>EFI_MTFTP6_OPCODE_WRQ</i> for a write request.
<i>EFI_MTFTP6_OACK_HEADER</i>	Filename	The file name to be downloaded or uploaded.
	OpCode	For this packet type <i>OpCode</i> = <i>EFI_MTFTP6_OPCODE_OACK</i> .
<i>EFI_MTFTP6_DATA_HEADER</i>	Data	The option strings in the option acknowledgement packet.
	OpCode	For this packet type <i>OpCode</i> = <i>EFI_MTFTP6_OPCODE_DATA</i> .
	Block	Block number of this data packet.
<i>EFI_MTFTP6_ACK_HEADER</i>	Data	The content of this data packet.
	OpCode	For this packet type <i>OpCode</i> = <i>EFI_MTFTP6_OPCODE_ACK</i> .
<i>EFI_MTFTP6_DATA8_HEADER</i>	Block	The block number of the data packet that is being acknowledged.
	OpCode	This field should be ignored and treated as reserved. For this packet type, <i>OpCode</i> = <i>EFI_MTFTP6_OPCODE_DATA8</i> .

continues on next page

Table 30.30 – continued from previous page

Block		
		This field should be ignored and treated as reserved. The block number of data packet.
Data		
		This field should be ignored and treated as reserved. The content of this data packet.
<i>EFI_MTFTP6_ACK8_HEADER</i>	OpCode	For this packet type <i>OpCode</i> = <i>EFI_MTFTP6_OPCODE_ACK8</i> .
	Block	The block number of the data packet that is being
<i>EFI_MTFTP6_ERROR_HEADER</i>	OpCode	For this packet type <i>OpCode</i> = <i>EFI_MTFTP6_OPCODE_ERROR</i> .
	ErrorCode	The error number as defined by the MTFTPv6 packet error codes. Values for <i>ErrorCode</i> are defined below.
	ErrorMessage	Error message string.

```
//
// MTFTP Packet OpCodes
//
#define EFI_MTFTP6_OPCODE_RRQ    1
#define EFI_MTFTP6_OPCODE_WRQ    2
#define EFI_MTFTP6_OPCODE_DATA    3
#define EFI_MTFTP6_OPCODE_ACK    6
#define EFI_MTFTP6_OPCODE_ERROR  5
#define EFI_MTFTP6_OPCODE_OACK    6
#define EFI_MTFTP6_OPCODE_DIR    7
//This field should be ignored and treated as reserved.
#define EFI_MTFTP4_OPCODE_DATA8    8
//This field should be ignored and treated as reserved.
#define EFI_MTFTP4_OPCODE_ACK8    9
```

Following is a description of the fields in the above definition.

Table 30.31: MTFTP Packet OpCode Descriptions

MTFTP Packet OpCode	Description
<i>EFI_MTFTP6_OPCODE_RRQ</i>	The MTFTPv6 packet is a read request.
<i>EFI_MTFTP6_OPCODE_WRQ</i>	The MTFTPv6 packet is a write request.
<i>EFI_MTFTP6_OPCODE_DATA</i>	The MTFTPv6 packet is a data packet.
<i>EFI_MTFTP6_OPCODE_ACK</i>	The MTFTPv6 packet is an acknowledgement packet.
<i>EFI_MTFTP6_OPCODE_ERROR</i>	The MTFTPv6 packet is an error packet.
<i>EFI_MTFTP6_OPCODE_OACK</i>	The MTFTPv6 packet is an option acknowledgement packet.
<i>EFI_MTFTP6_OPCODE_DIR</i>	The MTFTPv6 packet is a directory query packet.
<i>EFI_MTFTP6_OPCODE_DATA8</i>	
	This field should be ignored and treated as reserved. The MTFTPv6 packet is a data packet with a big block number.

continues on next page

Table 30.31 – continued from previous page

EFI_MTF TP6_OPCODE_ACK8

This field should be ignored and treated as reserved.
 The MTF TPv6 packet is an acknowledgement packet with a big block number.

```
//
// MTF TP ERROR Packet ErrorCodes
//
#define EFI_MTF TP6_ERRORCODE_NOT_DEFINED      0
#define EFI_MTF TP6_ERRORCODE_FILE_NOT_FOUND  1
#define EFI_MTF TP6_ERRORCODE_ACCESS_VIOLATION 2
#define EFI_MTF TP6_ERRORCODE_DISK_FULL       3
#define EFI_MTF TP6_ERRORCODE_ILLEGAL_OPERATION 4
#define EFI_MTF TP6_ERRORCODE_UNKNOWN_TRANSFER_ID 5
#define EFI_MTF TP6_ERRORCODE_FILE_ALREADY_EXISTS 6
#define EFI_MTF TP6_ERRORCODE_NO_SUCH_USER    7
#define EFI_MTF TP6_ERRORCODE_REQUEST_DENIED  8
```

Table 30.32: MTF TP ERROR Packet ErrorCode Descriptions

MTF TP ERROR Packet ErrorCodes	Description
<i>EFI_MTF TP6_ERRORCODE_NOT_DEFINED</i>	The error code is not defined. See the error message in the packet (if any) for details.
<i>EFI_MTF TP6_ERRORCODE_FILE_NOT_FOUND</i>	The file was not found.
<i>EFI_MTF TP6_ERRORCODE_ACCESS_VIOLATION</i>	There was an access violation.
<i>EFI_MTF TP6_ERRORCODE_DISK_FULL</i>	The disk was full or its allocation was exceeded.
<i>EFI_MTF TP6_ERRORCODE_ILLEGAL_OPERATION</i>	The MTF TPv6 operation was illegal.
<i>EFI_MTF TP6_ERRORCODE_UNKNOWN_TRANSFER_ID</i>	The transfer ID is unknown.
<i>EFI_MTF TP6_ERRORCODE_FILE_ALREADY_EXISTS</i>	The file already exists.
<i>EFI_MTF TP6_ERRORCODE_NO_SUCH_USER</i>	There is no such user.
<i>EFI_MTF TP6_ERRORCODE_REQUEST_DENIED</i>	The request has been denied due to option negotiation.

Status Codes Returned

EFI_SUCCESS	An MTF TPv6 OACK packet was received and is in the <i>Packet</i> .
-------------	--

continues on next page

Table 30.33 – continued from previous page

EFI_INVALID_PARAMETER	One or more of the following conditions is TRUE: • <i>This</i> is NULL. • <i>Filename</i> is NULL. • <i>OptionCount</i> is not zero and <i>OptionList</i> is NULL. • One or more options in <i>OptionList</i> have wrong format. • <i>PacketLength</i> is NULL. • <i>OverrideData.ServerIp</i> is not a valid unicast IPv6 address and not set to all zero.
EFI_UNSUPPORTED	One or more options in the <i>OptionList</i> are unsupported by this implementation.
EFI_NOT_STARTED	The EFI MTFTPv6 Protocol driver has not been started.
EFI_NO_MAPPING	The underlying IPv6 driver was responsible for choosing a source address for this instance, but no source address was available for use.
EFI_ACCESS_DENIED	The previous operation has not completed yet.
EFI_OUT_OF_RESOURCES	Required system resources could not be allocated.
EFI_TFTP_ERROR	An MTFTPv6 ERROR packet was received and is in the <i>Packet</i> .
EFI_NETWORK_UNREACHABLE	An ICMP network unreachable error packet was received and the <i>Packet</i> is set to NULL .
EFI_NETWORK_UNREACHABLE	An ICMP host unreachable error packet was received and the <i>Packet</i> is set to NULL .
EFI_NETWORK_UNREACHABLE	An ICMP protocol unreachable error packet was received and the <i>Packet</i> is set to NULL .
EFI_NETWORK_UNREACHABLE	An ICMP port unreachable error packet was received and the <i>Packet</i> is set to NULL .
EFI_ICMP_ERROR	Some other ICMP ERROR packet was received and the <i>Packet</i> is set to NULL .
EFI_PROTOCOL_ERROR	An unexpected MTFTPv6 packet was received and is in the <i>Packet</i> .
EFI_TIMEOUT	No responses were received from the MTFTPv6 server.
EFI_DEVICE_ERROR	An unexpected network error or system error occurred.

30.4.2.5 EFI_MTFTP6_PROTOCOL.ParseOptions()

Summary

Parse the options in an MTFTPv6 OACK packet.

Prototype

```
typedef
EFI_STATUS
(EFI_API *EFI_MTFTP6_PARSE_OPTIONS)(
    IN EFI_MTFTP6_PROTOCOL      *This,
    IN UINT32                   PacketLen,
    IN EFI_MTFTP6_PACKET        *Packet,
    OUT UINT32                   *OptionCount,
    OUT EFI_MTFTP6_OPTION       **OptionList OPTIONAL
);
```

Parameters

This

Pointer to the *EFI_MTFTP6_PROTOCOL* instance.

PacketLen

Length of the OACK packet to be parsed.

Packet

Pointer to the OACK packet to be parsed. Type *EFI_MTFTP6_PACKET* is defined in *EFI_MTFTP6_PROTOCOL.GetInfo()*.

OptionCount

Pointer to the number of options in the following *OptionList*.

OptionList

Pointer to *EFI_MTFTP6_OPTION* storage. Each pointer in the *OptionList* points to the corresponding MTFTP option buffer in the *Packet*. Call the EFI Boot Service *FreePool()* to release the *OptionList* if the options in this *OptionList* are not needed any more. Type *EFI_MTFTP6_OPTION* is defined in *EFI_MTFTP6_PROTOCOL.GetInfo()*.

Description

The *ParseOptions()* function parses the option fields in an MTFTPv6 OACK packet and returns the number of options that were found and optionally a list of pointers to the options in the packet.

If one or more of the option fields are not valid, then *EFI_PROTOCOL_ERROR* is returned and * *OptionCount* and * *OptionList* stop at the last valid option.

Status Codes Returned

EFI_SUCCESS	The OACK packet was valid and the <i>OptionCount</i> and <i>OptionList</i> parameters have been updated.
EFI_INVALID_PARAMETER	One or more of the following conditions is TRUE : • <i>PacketLen</i> is 0. • <i>Packet</i> is NULL or <i>Packet</i> is not a valid MTFTPv6 packet. • <i>OptionCount</i> is NULL.
EFI_NOT_FOUND	No options were found in the OACK packet.
EFI_OUT_OF_RESOURCES	Storage for the <i>OptionList</i> array can not be allocated.
EFI_PROTOCOL_ERROR	One or more of the option fields is invalid.

30.4.2.6 EFI_MTFTP6_PROTOCOL.ReadFile()

Summary

Download a file from an MTFTPv6 server.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_MTFTP6_READ_FILE)(
    IN EFI_MTFTP6_PROTOCOL      *This,
    IN EFI_MTFTP6_TOKEN         *Token
);
```

Parameters

This

Pointer to the *EFI_MTFTP6_PROTOCOL* instance.

Token

Pointer to the token structure to provide the parameters that are used in this operation. Type *EFI_MTFTP6_TOKEN* is defined in “Related Definitions” below.

Description

The *ReadFile()* function is used to initialize and start an MTFTPv6 download process and optionally wait for completion. When the download operation completes, whether successfully or not, the *Token.Status* field is updated by the EFI MTFTPv6 Protocol driver and then *Token.Event* is signaled if it is not **NULL**.

Data can be downloaded from the MTFTPv6 server into either of the following locations:

- A fixed buffer that is pointed to by *Token.Buffer*
- A download service function that is pointed to by *Token.CheckPacket*

If both *Token.Buffer* and *Token.CheckPacket* are used, then *Token.CheckPacket* will be called first. If the call is successful, the packet will be stored in *Token.Buffer*.

Related Definitions

```

//*****
// EFI_MTFTP6_TOKEN
//*****
typedef struct {
    EFI_STATUS          Status;
    EFI_EVENT           Event;
    EFI_MTFTP6_OVERRIDE_DATA OverrideData;
    UINT8               *Filename;
    UINT8               *ModeStr;
    UINT32              OptionCount;
    EFI_MTFTP6_OPTION   OptionList;
    UINT64              BufferSize;
    VOID                *Buffer;
    VOID                *Context;
    EFI_MTFTP6_CHECK_PACKET CheckPacket;
    EFI_MTFTP6_TIMEOUT_CALLBACK TimeoutCallback;
    EFI_MTFTP6_PACKET_NEEDED PacketNeeded;
} EFI_MTFTP6_TOKEN;
    
```

Status

The status that is returned to the caller at the end of the operation to indicate whether this operation completed successfully. Defined *Status* values are listed below.

Event

The event that will be signaled when the operation completes. If set to **NULL**, the corresponding function will wait until the read or write operation finishes. The type of *Event* must be *EVT_NOTIFY_SIGNAL*.

OverrideData

If not **NULL**, the data that will be used to override the existing configure data. Type *EFI_MTFTP6_OVERRIDE_DATA* is defined in *EFI_MTFTP6_PROTOCOL* .GetInfo().

Filename

Pointer to the null-terminated ASCII file name string.

ModeStr

Pointer to the null-terminated ASCII mode string. If **NULL**, octet is used.

OptionCount

Number of option/value string pairs.

OptionList

Pointer to an array of option/value string pairs. Ignored if *OptionCount* is zero. Both a remote server and this driver implementation should support these options. If one or more options are unrecognized by this implementation, it is sent to the remote server without being changed. Type *EFI_MTFTP6_OPTION* is defined in *EFI_MTFTP6_PROTOCOL*.GetInfo().

BufferSize

On input, the size, in bytes, of *Buffer*. On output, the number of bytes transferred.

Buffer

Pointer to the data buffer. Data that is downloaded from the MTFTPv6 server is stored here. Data that is uploaded to the MTFTPv6 server is read from here. Ignored if *BufferSize* is zero.

Context

Pointer to the context that will be used by *CheckPacket*, *TimeoutCallback* and *PacketNeeded*.

CheckPacket

Pointer to the callback function to check the contents of the received packet. Type *EFI_MTFTP6_CHECK_PACKET* is defined below.

TimeoutCallback

Pointer to the function to be called when a timeout occurs. Type *EFI_MTFTP6_TIMEOUT_CALLBACK* is defined below.

PacketNeeded

Pointer to the function to provide the needed packet contents. Only used in *WriteFile()* operation. Type *EFI_MTFTP6_PACKET_NEEDED* is defined below.

The *EFI_MTFTP6_TOKEN* structure is used for both the MTFTPv6 reading and writing operations.

The caller uses this structure to pass parameters and indicate the operation context. After the reading or writing operation completes, the EFI MTFTPv6 Protocol driver updates the Status parameter and the Event is signaled if it is not **NULL**. The following table lists the status codes that are returned in the Status parameter.

Status Codes Returned in the Status Parameter

<i>EFI_SUCCESS</i>	The data file has been transferred successfully.
<i>EFI_OUT_OF_RESOURCES</i>	Required system resources could not be allocated.
<i>EFI_BUFFER_TOO_SMALL</i>	<i>BufferSize</i> is not zero but not large enough to hold the downloaded data in downloading process.
<i>EFI_ABORTED</i>	Current operation is aborted by user.
<i>EFI_NETWORK_UNREACHABLE</i>	An ICMP network unreachable error packet was received.
<i>EFI_NETWORK_UNREACHABLE</i>	An ICMP host unreachable error packet was received.
<i>EFI_NETWORK_UNREACHABLE</i>	An ICMP protocol unreachable error packet was received.
<i>EFI_NETWORK_UNREACHABLE</i>	An ICMP port unreachable error packet was received.
<i>EFI_ICMP_ERROR</i>	Some other ICMP ERROR packet was received.
<i>EFI_TIMEOUT</i>	No responses were received from the MTFTPv6 server.
<i>EFI_TFTP_ERROR</i>	An MTFTPv6 ERROR packet was received.
<i>EFI_DEVICE_ERROR</i>	An unexpected network error or system error occurred.

```

//*****
// EFI_MTFTP6_CHECK_PACKET
//*****
typedef
EFI_STATUS
(EFIAPI *EFI_MTFTP6_CHECK_PACKET)(
    IN EFI_MTFTP6_PROTOCOL      *This,
    IN EFI_MTFTP6_TOKEN         *Token,
    IN UINT16                   PacketLen,
    IN EFI_MTFTP6_PACKET        *Packet
);

```

This

Pointer to the *EFI_MTFTP6_PROTOCOL* instance.

Token

The token that the caller provided in the *EFI_MTFTP6_PROTOCOL.ReadFile()*, *WriteFile()* or *ReadDirectory()* function. Type *EFI_MTFTP6_TOKEN* is defined in *EFI_MTFTP6_PROTOCOL.ReadFile()*.

PacketLen

Indicates the length of the packet.

Packet

Pointer to an MTFTPv6 packet. Type *EFI_MTFTP6_PACKET* is defined in *EFI_MTFTP6_PROTOCOL.GetInfo()*.
EFI_MTFTP6_CHECK_PACKET is a callback function that is provided by the caller to intercept the *EFI_MTFTP6_OPCODE_DATA* or *EFI_MTFTP6_OPCODE_DATA8* packets processed in the *EFI_MTFTP6_PROTOCOL.ReadFile()* function, and alternatively to intercept *EFI_MTFTP6_OPCODE_OACK* or *EFI_MTFTP6_OPCODE_ERROR* packets during a call to *EFI_MTFTP6_PROTOCOL.ReadFile()*, *WriteFile()* or *ReadDirectory()*. Whenever an MTFTPv6 packet with the type described above is received from a server, the EFI MTFTPv6 Protocol driver will call *EFI_MTFTP6_CHECK_PACKET* function to let the caller have an opportunity to process this packet. Any status code other than *EFI_SUCCESS* that is returned from this function will abort the transfer process.

```

//*****
// EFI_MTFTP6_TIMEOUT_CALLBACK
//*****
typedef
EFI_STATUS
(EFIAPI *EFI_MTFTP6_TIMEOUT_CALLBACK)(
    IN EFI_MTFTP6_PROTOCOL      *This,
    IN EFI_MTFTP6_TOKEN         *Token
);

```

This

Pointer to the *EFI_MTFTP6_PROTOCOL* instance.

Token

The token that is provided in the *EFI_MTFTP6_PROTOCOL.ReadFile()* or *EFI_MTFTP6_PROTOCOL.ReadDirectory()* functions by the caller. Type *EFI_MTFTP6_TOKEN* is defined in

EFI_MTFTP6_PROTOCOL.ReadFile().

EFI_MTFTP6_TIMEOUT_CALLBACK is a callback function that the caller provides to capture the timeout event in the *EFI_MTFTP6_PROTOCOL.ReadFile()*, *EFI_MTFTP6_PROTOCOL.WriteFile()* or *EFI_MTFTP6_PROTOCOL.ReadDirectory()* functions. Whenever a timeout occurs, the EFI MTFTPv6 Protocol driver will call the *EFI_MTFTP6_TIMEOUT_CALLBACK* function to notify the caller of the timeout event. Any status code other than *EFI_SUCCESS* that is returned from this function will abort the current download process.

```
//*****
// EFI_MTFTP6_PACKET_NEEDED
//*****
typedef
EFI_STATUS
(EFIAPI *EFI_MTFTP6_PACKET_NEEDED)(
    IN EFI_MTFTP6_PROTOCOL      *This,
    IN EFI_MTFTP6_TOKEN         Token,
    IN OUT UINT16               *Length,
    OUT VOID                    **Buffer
);
```

This

Pointer to the *EFI_MTFTP6_PROTOCOL* instance.

Token

The token provided in the *EFI_MTFTP6_PROTOCOL.WriteFile()* by the caller.

Length

Indicates the length of the raw data wanted on input, and the length the data available on output.

Buffer

Pointer to the buffer where the data is stored.

EFI_MTFTP6_PACKET_NEEDED is a callback function that the caller provides to feed data to the *EFI_MTFTP6_PROTOCOL.WriteFile()* function. *EFI_MTFTP6_PACKET_NEEDED* provides another mechanism for the caller to provide data to upload other than a static buffer. The EFI MTFTP6 Protocol driver always calls *EFI_MTFTP6_PACKET_NEEDED* to get packet data from the caller if no static buffer was given in the initial call to *EFI_MTFTP6_PROTOCOL.WriteFile()* function. Setting ***Length** to zero signals the end of the session. Returning a status code other than *EFI_SUCCESS* aborts the session.

Status Codes Returned

EFI_SUCCESS	The data file is being downloaded.
EFI_INVALID_PARAMETER	One or more of the parameters is not valid. • <i>This</i> is NULL. • <i>Token</i> is NULL. • <i>Token.Filename</i> is NULL. • <i>Token.OptionCount</i> is not zero and <i>Token.OptionList</i> is NULL. • One or more options in <i>Token.OptionList</i> have wrong format. • <i>Token.Buffer</i> and <i>Token.CheckPacket</i> are both NULL. • <i>Token.OverrideData.ServerIp</i> is not a valid unicast IPv6 address and not set to all zero.

continues on next page

Table 30.36 – continued from previous page

EFI_UNSUPPORTED	One or more options in the <i>Token.OptionList</i> are not supported by this implementation.
EFI_NOT_STARTED	The EFI MTFTPv6 Protocol driver has not been started.
EFI_NO_MAPPING	The underlying IPv6 driver was responsible for choosing a source address for this instance, but no source address was available for use.
EFI_ALREADY_STARTED	This <i>Token</i> is being used in another MTFTPv6 session.
EFI_ACCESS_DENIED	The previous operation has not completed yet.
EFI_OUT_OF_RESOURCES	Required system resources could not be allocated.
EFI_DEVICE_ERROR	An unexpected network error or system error occurred.
EFI_NO_MEDIA	There was a media error.

30.4.2.7 EFI_MTFTP6_PROTOCOL.WriteFile()

Summary

Send a file to an MTFTPv6 server. May be unsupported in some implementations.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_MTFTP6_WRITE_FILE)(
    IN EFI_MTFTP6_PROTOCOL      *This,
    IN EFI_MTFTP6_TOKEN         *Token
);
```

Parameters

This

Pointer to the *EFI_MTFTP6_PROTOCOL* instance.

Token

Pointer to the token structure to provide the parameters that are used in this function. Type *EFI_MTFTP6_TOKEN* is defined in *EFI_MTFTP6_PROTOCOL.ReadFile()*.

Description

The *WriteFile()* function is used to initialize an uploading operation with the given option list and optionally wait for completion. If one or more of the options is not supported by the server, the unsupported options are ignored and a standard TFTP process starts instead. When the upload process completes, whether successfully or not, *Token.Event* is signaled, and the EFI MTFTPv6 Protocol driver updates *Token.Status*.

The caller can supply the data to be uploaded in the following two modes:

- Through the user-provided buffer
- Through a callback function

With the user-provided buffer, the *Token.BufferSize* field indicates the length of the buffer, and the driver will upload the data in the buffer. With an *EFI_MTFTP6_PACKET_NEEDED* callback function, the driver will call this callback function to get more data from the user to upload. See the definition of *EFI_MTFTP6_PACKET_NEEDED* for more information. These two modes cannot be used at the same time. The callback function will be ignored if the user provides the buffer.

Status Codes Returned

EFI_SUCCESS	The upload session has started.
EFI_UNSUPPORTED	The operation is not supported by this implementation.
EFI_INVALID_PARAMETER	One or more of the following conditions is TRUE : • <i>This</i> is NULL. • <i>Token</i> is NULL. • <i>Token.Filename</i> is NULL. • <i>Token.OptionCount</i> is not zero and <i>Token.OptionList</i> is NULL. • One or more options in <i>Token.OptionList</i> have wrong format. • <i>Token.Buffer</i> and <i>Token.PacketNeeded</i> are both NULL. • <i>Token.OverrideData.ServerIp</i> is not a valid unicast IPv6 address and not set to all zero.
EFI_UNSUPPORTED	One or more options in the <i>Token.OptionList</i> are not supported by this implementation.
EFI_NOT_STARTED	The EFI MTFTPv6 Protocol driver has not been started.
EFI_NO_MAPPING	The underlying IPv6 driver was responsible for choosing a source address for this instance, but no source address was available for use.
EFI_ALREADY_STARTED	This <i>Token</i> is already being used in another MTFTPv6 session.
EFI_OUT_OF_RESOURCES	Required system resources could not be allocated.
EFI_ACCESS_DENIED	The previous operation has not completed yet.
EFI_DEVICE_ERROR	An unexpected network error or system error occurred.
EFI_NO_MEDIA	There was a media error.

30.4.2.8 EFI_MTFTP6_PROTOCOL.ReadDirectory()

Summary

Download a data file directory from an MTFTPv6 server. May be unsupported in some implementations.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_MTFTP6_READ_DIRECTORY)(
    IN EFI_MTFTP6_PROTOCOL          *This,
    IN EFI_MTFTP6_TOKEN             *Token
);
```

Parameters

This

Pointer to the *EFI_MTFTP6_PROTOCOL* instance.

Token

Pointer to the token structure to provide the parameters that are used in this function. Type *EFI_MTFTP6_TOKEN* is defined in *EFI_MTFTP6_PROTOCOL.ReadFile()*.

Description

The *ReadDirectory()* function is used to return a list of files on the MTFTPv6 server that are logically (or operationally) related to *Token.Filename*. The directory request packet that is sent to the server is built with the option list that was provided by caller, if present.

The file information that the server returns is put into either of the following locations:

- A fixed buffer that is pointed to by *Token.Buffer*
- A download service function that is pointed to by *Token.CheckPacket*

If both *Token.Buffer* and *Token.CheckPacket* are used, then *Token.CheckPacket* will be called first. If the call is successful, the packet will be stored in *Token.Buffer*.

The returned directory listing in the *Token.Buffer* or *EFI_MTFTP6_PACKET* consists of a list of two or three variable-length ASCII strings, each terminated by a null character, for each file in the directory. If the multicast option is involved, the first field of each directory entry is the static multicast IP address and UDP port number that is associated with the file name. The format of the field is *ip:ip:ip:ip:port*. If the multicast option is not involved, this field and its terminating null character are not present.

The next field of each directory entry is the file name and the last field is the file information string. The information string contains the file size and the create/modify timestamp. The format of the information string is *filesize yyyy-mm-dd hh:mm:ss:ffff*. The timestamp is Coordinated Universal Time (UTC; also known as Greenwich Mean Time [GMT]).

Status Codes Returned

EFI_SUCCESS	The MTFTPv6 related file “directory” has been downloaded.
EFI_UNSUPPORTED	The EFI MTFTPv6 Protocol driver does not support this function.
EFI_INVALID_PARAMETER	One or more of these conditions is TRUE: • <i>This</i> is NULL. • <i>Token</i> is NULL. • <i>Token.Filename</i> is NULL. • <i>Token.OptionCount</i> is not zero and <i>Token.OptionList</i> is NULL. • One or more options in <i>Token.OptionList</i> have wrong format. • <i>Token.Buffer</i> and <i>Token.CheckPacket</i> are both NULL. • <i>Token.OverrideData.ServerIp</i> is not a valid unicast IPv6 address and not set to all zero.
EFI_UNSUPPORTED	One or more options in the <i>Token.OptionList</i> are not supported by this implementation.
EFI_NOT_STARTED	The EFI MTFTPv6 Protocol driver has not been started.
EFI_NO_MAPPING	The underlying IPv6 driver was responsible for choosing a source address for this instance, but no source address was available for use.
EFI_ALREADY_STARTED	This <i>Token</i> is already being used in another MTFTPv6 session.
EFI_OUT_OF_RESOURCES	Required system resources could not be allocated.
EFI_ACCESS_DENIED	The previous operation has not completed yet.
EFI_DEVICE_ERROR	An unexpected network error or system error occurred.

30.4.2.9 EFI_MTFTP6_PROTOCOL.Poll()

Summary

Polls for incoming data packets and processes outgoing data packets.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_MTFTP6_POLL) (
    IN EFI_MTFTP6_PROTOCOL    *This
);
```

Parameters

This

Pointer to the *EFI_MTFTP6_PROTOCOL* instance.

Description

The *Poll()* function can be used by network drivers and applications to increase the rate that data packets are moved between the communications device and the transmit and receive queues.

In some systems, the periodic timer event in the managed network driver may not poll the underlying communications device fast enough to transmit and/or receive all data packets without missing incoming packets or dropping outgoing packets. Drivers and applications that are experiencing packet loss should try calling the *Poll()* function more often.

Status Codes Returned

EFI_SUCCESS	Incoming or outgoing data was processed.
EFI_NOT_STARTED	This EFI MTFTPv6 Protocol instance has not been started.
EFI_INVALID_PARAMETER	<i>This</i> is NULL .
EFI_DEVICE_ERROR	An unexpected system or network error occurred.
EFI_TIMEOUT	Data was dropped out of the transmit and/or receive queue. Consider increasing the polling rate.